Cairo University
Faculty of Computing and Artificial Intelligence
Departments of AI and CS

TestPilot

An Autonomous Web Testing Agent

Supervised by

Prof. Basher Abdelfattah

TA. Muhammed Atta

Team Members

20220373	Hesham Abdelnaser Ali
20221151	Mostafa Mahmoud Hosni
20220212	Ali Raouf Muhammed

Graduation Project Final Document

Academic Year 2025/2026

Table of Content

Table of Content **2**
List of Tables **5**
List of Figures **6**
List of Abbreviations **7**
Abstract **8**
ملخص **9**
Chapter 1: Introduction **1**
1.1 Background and Motivation 1
1.2 Problem Definition 2
1.3 Project Objectives and Proposed Solution 2
1.4 Project Scope and Limitation 4
1.5 Development Methodology 4
1.6 Tools and Technologies Used 5
1.6.1 Software Tools 5
1.6.2 Hardware Requirements 6
1.7 Project Timeline 6
1.8 Report Organization 8
Chapter 2: Related Work **10**
2.1 Overview of Existing Research 10
2.1.1 Scripted Test Automation and Selector Fragility 10
2.1.2 Autonomous Web Agents 11
2.1.3 Autonomous Test Agents 11
2.2 Comparative Analysis 12
2.2.1 Input Formats and Step Orchestration 13
2.2.2 Assertion Verification and Observability 14
2.2.3 Error Recovery and Reproducibility 14
Chapter 3: System Analysis **16**
3.1 Stakeholder Analysis 16
3.2 Functional Requirements 17
3.2.1 Test and Project Management 17
3.2.2 Automated Execution and Orchestration 18
3.2.3 Data Streaming and Historical Reporting 19
3.3 Non-Functional Requirements 20
3.3.1 Performance and Latency 20
3.3.2 Security and Data Protection 21
3.3.3 Usability and Human Factors 21

3.3.4 Compatibility and Portability....21
3.4 Use Case Diagram....22
3.4.1 Actor Roles and Responsibilities....23
3.4.2 Use Case Specifications and the Include Relationship....23
3.5 Feasibility Study....24
Chapter 4: System Design....26
4.1 System Architecture....26
4.1.1 Overview of the Architecture....26
4.1.2 Presentation Layer....28
3.1.3 API and Service Layer....29
4.1.4 Orchestration Layer....29
4.1.5 Execution and Automation Layer....32
4.1.6 Persistence Layer....32
4.2 Class Diagrams....33
4.2.1 Service and Event Infrastructure....33
4.2.2 Agent Orchestration and Execution....35
4.2.3 Domain and Persistence Mapping....36
4.3 Sequence Diagrams....38
4.3.1 Test Execution Initialization and Session Establishment....38
4.3.2 The Core Agent Planning and Tool Execution Loop....40
4.3.3 Step Verification, Failure Analysis, and Self-Healing Recovery....42
4.4 Database Entity-Relationship Diagram (ERD)....44
4.5 User Interface Wireframes / Mockups....47
4.5.1 Projects Dashboard View....47
4.5.2 Interactive Test Workspace View....48
4.5.3 Historical Run Report View....50
4.6 Security Design Considerations....52
4.6.1 API Key Lifecycle and In-Memory Containment....52
4.6.2 Injection Prevention and Server-Side Validation....52
4.6.3 Execution Context and Session Isolation....53
4.6.4 Asynchronous Task and Process Containment....53
Chapter 5: Implementation....54
5.1 Implementation Environment and Setup....54
5.1.1 Operating System and Environment Sandbox....54
5.1.2 Backend Runtime and Dependency Configurations....54
5.1.3 Frontend Runtime and Dependency Configurations....55
5.2 Key Implementation Details....56
5.2.1 Element Discovery and Set-of-Marks Overlay Algorithm....56
5.2.2 DOM Stability Detection Algorithm....57
5.2.3 Major Engineering Decisions....57
5.3 Output Samples and Data Payloads....58

5.3.1 JSON Test Case Portability Payload 58
5.3.2 WebSocket Event Frame Payload 60
5.3.3 Raw Execution Log Output 60
Chapter 6: Testing and Evaluation 62
6.1 Evaluation Setup and Test Suite 62
6.3 Design-Level Baseline Comparison 63
6.4 Threats to Validity 64
Chapter 7: Conclusion and Future Work 66
7.1 Summary of Achievements 66
7.2 Lessons Learned 67
7.3 Future Enhancements 67
References 69

List of Tables

Table 1.1: Summary of the Project Roadmap 8
Table 2.1: Comparison of TestPilot with related autonomous test agents and commercial AI-assisted testing platforms. 14
Table 6.1: SauceLabs Test Cases Suite 67
Table 6.2: Design Comparison Between TestPilot and Baselines 68

List of Figures

Figure	Page Number
Figure 3.1: Use Case Diagram	25
Figure 4.1: System Architecture Diagram	30
Figure 4.2: Agent Architecture and Execution Flow	33
Figure 4.3: Service and Event infrastructure	37
Figure 4.4: Agent Orchestration and Execution	39
Figure 4.5: Domain and Persistence Mapping	40
Figure 4.6: Test Execution Initialization and Session Establishment Sequence Diagram	42
Figure 4.7: The Core Agent Planning and Tool Execution Loop Sequence Diagram	44
Figure 4.8: Step Verification, Failure Analysis, and Self-Healing Recovery Sequence Diagram	46
Figure 4.9: Database Entity-Relationship Diagram	48
Figure 4.10: Projects Dashboard View	51
Figure 4.11: Interactive Test Workspace View	52
Figure 4.12: Historical Run Report View	53

List of Abbreviations

Abbreviations	Definition
DOM	Document Object Model
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
XML	eXtensible Markup Language
XPath	XML Path Language
JSON	JavaScript Object Notation
SQL	Structured Query Language
API	Application Programming Interface
LLM	Large Language Model
SoM	Set-of-Mark
ATA	Autonomous Test Agent
QA	Quality Assurance
UUID	Universally Unique Identifier
UML	Unified Modeling Language
SPA	Single-Page web Application
ORM	Object-Relational Mapper
ERD	Entity-Relationship Diagram

Abstract

Software testing consumes a significant share of engineering resources across the software development lifecycle. In web applications, conventional automation frameworks, such as Selenium and Playwright, rely on scripts that locate elements through DOM dependent strategies, including CSS selectors or XPath. These locators frequently break after user interface refactoring, which increases maintenance work and reduces test reliability.

This report presents TestPilot, an autonomous web testing agent designed to execute end-to-end acceptance tests written in natural language. Instead of relying on pre-written element selectors for each step, TestPilot observes the current page state, determines the next action, executes it in browser, and verifies the expected outcome before proceeding.

The core logic of TestPilot is implemented as an observe-reason-act loop using a LangGraph state machine. TestPilot uses Playwright to drive a headless Chromium browser. It implements a Set-of-Marks visual prompting technique by injecting JavaScript into the page to label interactive elements with numerical identifiers. This allows the underlying LLM to reference elements without relying on fragile DOM pathways. The backend is built with Python and FastAPI, storing test metadata in a SQLite database via SQLAlchemy. A web frontend developed with React, Tailwind CSS, and Vite connects to the backend, enabling users to edit test cases and monitor execution traces in real time via WebSockets.

ملخص

يستهلك اختبار البرمجيات حصة كبيرة من موارد الهندسة على مدار دورة حياة تطوير البرمجيات. في تطبيقات الويب، تعتمد أطر العمل التقليدية للأتمتة، مثل Selenium و Playwright، على نصوص برمجية تحدد موقع العناصر من خلال استراتيجيات تعتمد على نموذج DOM (Document Object Model)، بما في ذلك محددات CSS أو XPath. وكثيرًا ما تتعطل هذه المحددات بعد إعادة هيكلة واجهة المستخدم، مما يزيد من أعباء الصيانة ويقلل من موثوقية الاختبارات.

يقدم هذا التقرير TestPilot، وهو عميل اختبار ويب مستقل (autonomous) مصمم لتنفيذ اختبارات القبول من النوع End-to-End المكتوبة باللغة الطبيعية. وبدلًا من الاعتماد على محددات عناصر مكتوبة مسبقًا لكل خطوة، يقوم TestPilot بمراقبة حالة الصفحة الحالية، وتحديد الإجراء التالي، وتنفيذه داخل المتصفح، والتحقق من النتيجة المتوقعة قبل الاستمرار.

يُنفَّذ المنطق الأساسي لـ TestPilot في صورة حلقة "مراقبة - استدلال - تنفيذ" (observe-reason-act) باستخدام آلة حالات (state machine) من LangGraph. يستخدم TestPilot مكتبة Playwright لتشغيل متصفح Chromium بدون واجهة رسومية (headless). ويطبّق تقنية التوجيه البصري المعروفة باسم "مجموعة العلامات" (Set-of-Marks) من خلال حقن كود JavaScript في الصفحة لوضع معرّفات رقمية على العناصر التفاعلية. وهذا يتيح لنموذج اللغة الكبير الأساسي (LLM) الإشارة إلى العناصر دون الاعتماد على مسارات DOM الهشّة. وتم بناء الواجهة الخلفية (backend) باستخدام Python و FastAPI، مع تخزين بيانات وصفية للاختبارات (metadata) في قاعدة بيانات SQLite عبر SQLAlchemy. وتتصل واجهة أمامية للويب (frontend) مطوَّرة باستخدام React و Tailwind CSS و Vite بالواجهة الخلفية، مما يتيح للمستخدمين تحرير حالات الاختبار ومراقبة آثار التنفيذ (execution traces) في الوقت الفعلي عبر WebSockets.

Chapter 1: Introduction

This chapter outlines the foundation for TestPilot, an autonomous agent designed to overcome the fragility of conventional DOM-dependent web testing by utilizing multimodal LLMs and Set-of-Marks (SoM) visual prompting for natural language test execution. It establishes the project's objectives, defining a structured approach to action planning, verification, and visual grounding, while outlining the scope of this browser-based, end-to-end testing solution.

1.1 Background and Motivation

Software development lifecycles depend on automated testing to verify application behavior before deployment. End-to-end testing validates user-visible workflows by simulating actual user interactions within a web browser. Conventional automation frameworks, such as Selenium, Playwright, and Cypress, require test scripts that target specific elements in the DOM. These scripts locate input fields, buttons, and links using locators like XML Path Language (XPath) queries or Cascading Style Sheets (CSS) selectors.

These locators capture the structural implementation details of a web page rather than its functional properties. When developers refactor a user interface (for example, by changing class names, reorganizing component trees, or introducing wrapper elements), the hardcoded selectors fail. This creates test fragility. Automated test runs fail despite the application continuing to function correctly from a user perspective.

Maintaining broken test locators represents a significant recurring expense for software engineering teams [16]. Automated suites require constant manual intervention to repair broken paths, which reduces the reliability of continuous integration pipelines.

Recent developments in multimodal Large Language Models (LLMs) present an alternative approach. Instead of using hardcoded selector strings, an agent can inspect visual screenshots and structural data to interact with the interface, mimicking how manual human testers verify applications. This approach reduces the dependency on DOM structures and minimizes test maintenance overhead.

1.2 Problem Definition

To reduce test script maintenance, web test automation must transition away from fixed, DOM-dependent selectors. The objective is to execute end-to-end acceptance tests written in natural language directly on a web application.

Existing autonomous web agents are designed for general task completion, such as booking a flight or purchasing an item [23]. These systems evaluate success based on final goal completion and can alter their path freely. Acceptance testing imposes strict structural constraints that generalist agents cannot handle. An acceptance test case requires a precise sequence of steps, intermediate validation assertions at each step, and explicit stopping conditions. If an intermediate assertion fails, the test must halt immediately and record the exact location and cause of the failure.

This requirement introduces three distinct technical challenges:

1. Grounding: Translating a natural language action description (such as "click the login button") to a specific coordinate or DOM element on a dynamic, visual screen.
2. Verification: Deciding whether an expected outcome (such as "the user profile page is displayed") is satisfied by inspecting the page state.
3. Error Diagnosis: Distinguishing an application regression from an agent execution mistake (such as misidentifying an element or misinterpreting a page state). This requires step-level trace outputs with supporting visual evidence.

1.3 Project Objectives and Proposed Solution

To address these challenges, this project designs and implements TestPilot, an autonomous web testing agent that executes acceptance tests written as ordered, natural language steps.

TestPilot uses a layered architecture to separate orchestrating, reasoning, executing, and persisting. The core agent reasoning loop runs on a LangGraph state machine, which manages sequential steps, handles execution retries, and coordinates transitions between state nodes.

For visual and structural observation, TestPilot implements a SoM visual prompting technique. It injects a JavaScript module into the browser via Playwright to identify interactive elements (such as buttons, links, and inputs) and draw numbered overlay boundaries on a page screenshot. This screenshot is sent to a multimodal LLM, which allows the model to select targets by their numerical markers instead of raw DOM selectors.

To ensure assertion verification is handled independently, the system routes verification tasks to a dedicated verification node (the assertor) before proceeding to the next step.

The backend is built with Python and FastAPI. It exposes REST endpoints to manage test cases and uses WebSockets to stream execution logs and visual states to a web client. The persistence layer saves historical execution traces to a local SQLite database using SQLAlchemy. A web frontend developed with React, Tailwind CSS, and Vite connects to the backend, enabling users to edit test cases and monitor execution traces in real time.

The concrete project objectives are to:

1. Design and build an agent to execute natural language web tests sequentially without hardcoded selectors.
2. Implement a visual grounding system using a dynamic element-labeling overlay.
3. Separate action planning, execution, and verification into distinct state machine nodes to improve execution accuracy.
4. Implement backend and a responsive web application to edit test cases and monitor execution traces in real time.
5. Evaluate system performance on a reproducible, offline benchmark to compare verdict accuracy against existing academic baselines.

1.4 Project Scope and Limitation

The boundary of TestPilot is defined by browser-based, end-to-end acceptance testing of web applications. The system accepts sequential, natural language test cases and executes them by interacting directly with HTML elements visible on Chromium browser instances. This process is

driven by Playwright and orchestrated by a state machine that handles step-by-step verification and retry operations. All project data, test cases, and historical run logs are stored in a local SQLite database, which is accessed and managed via a local web interface built with React, Vite, and Tailwind CSS.

Conversely, several constraints define the limits of this system. TestPilot does not support native mobile or desktop software applications. Furthermore, the testing capabilities are limited to visual and structural elements exposed on the browser viewport; low-level unit testing, API contract verification, performance load testing, and security penetration tests are outside the scope of this work. Because the execution logic relies on external LLM APIs, network latency and API downtime directly impact execution speeds. Additionally, the element-labeling module requires interactive elements to be correctly represented in the browser accessibility tree. If custom graphical components lack standard accessibility tags, they may not be labeled correctly by the JavaScript injection layer.

1.5 Development Methodology

TestPilot was developed using an iterative engineering lifecycle, which allowed the team to construct and refine individual architectural layers in sequential cycles. The process began with requirement analysis and literature research, where the team reviewed publications on web test agents and analyzed the baseline architectures of SeeAct-ATA and PinATA [1].

Following this research, the team entered the architecture and design phase, defining the boundaries of the five system layers, constructing the relational database schema, and plotting the state transitions for the LangGraph orchestrator. This led directly to the core development phase. During this period, the FastAPI service and the Playwright browser controller were programmed, and the logic for the agent nodes was implemented. Simultaneously, the visual grounding layer was built by writing the SoM JavaScript element-labeling script.

In the final phases, the team focused on frontend creation and benchmarking. We built the web client using React, Vite, and Tailwind CSS to enable test editing and real-time monitoring via WebSocket connections.

1.6 Tools and Technologies Used

1.6.1 Software Tools

The backend infrastructure of TestPilot is built on Python 3.13, which provides the necessary libraries for asynchronous operations and language model integration. Web services and endpoint management are handled by FastAPI 0.128. This framework manages REST API routing and maintains persistent, bidirectional WebSocket connections for real-time data streaming. The core execution flow is orchestrated using LangGraph v1.0, which treats the test execution loop as a stateful directed graph, tracking variables and transition rules across different steps.

Browser manipulation and page observation are managed via Playwright 1.55. Playwright drives headless Chromium instances, executes custom JavaScript injections for element labeling, and captures page screenshots. For data storage, TestPilot uses an SQLite 3 database engine. This relational database stores project metadata, test definitions, and run histories, and is accessed through SQLAlchemy 2.0 to ensure database operations are secure and parameterized.

The user interface is developed as an SPA using React 18 and styled with Tailwind CSS 3.4. The application is compiled and served using the Vite 8 build tool, which connects to the backend to display visual states and run progress. Language processing and visual reasoning are driven by external API integrations.

1.6.2 Hardware Requirements

The hardware configuration used to develop and evaluate this system consists of a standard development workstation. This machine is equipped with an Intel Core i5-10200H processor (as featured in the Dell G15 5510), 16 gigabytes of random-access memory (RAM), and a solid-state drive. This hardware setup is sufficient to host the FastAPI server, maintain the local database, and

run multiple headless Chromium browser sessions simultaneously without degrading the performance of the React web client.

1.7 Project Timeline

The development roadmap of TestPilot was structured as a series of sequential and overlapping milestones distributed across the academic year. The project commenced with an initial research phase focused on analyzing existing literature in autonomous web testing. During this phase, the team studied the limitations of the SeeAct-ATA and PinATA baselines [1], while reviewing commercial platforms such as Applitools and Functionize. Following this analysis, the team finalized the technology stack by selecting LangGraph for agent orchestration, Playwright for browser automation, FastAPI for backend services, SQLite for persistence, and a frontend framework built on React, Tailwind CSS, and Vite.

Once the technological foundation was established, the project transitioned into requirement analysis and design. This phase involved defining the functional and non-functional specifications and generating the system Unified Modeling Language (UML) diagrams, including use case, class, and sequence diagrams, alongside the relational database entity-relationship diagram. Development of the browser automation layer followed, where the team wrote a Playwright wrapper to handle metadata enrichment, custom JavaScript injection, and page screenshot capture. This enabled the development of the core artificial intelligence agent, which implemented the LangGraph state machine and the observe-reason-act reasoning loop.

The subsequent milestones focused on backend service creation, database integration, and frontend implementation. The team developed the FastAPI application endpoints and WebSocket handlers to support real-time data streaming, while database persistence was integrated using SQLAlchemy models to store test cases, execution runs, and detailed step logs. Frontend dashboard development took place to construct the React interface for test authoring and real-time execution monitoring. The final stages involved system integration to connect the frontend, backend, and agent components, followed by compiling the final documentation, user manual, and thesis presentation.

The specific execution dates and development milestones are recorded in Table 1.1 below.

Task	Task Title	Description	Completion Date
1	Research	Reviewed publications on autonomous web agents, the WebArena benchmark, and commercial AI automation platforms.	Nov 2025
2	Tech Stack Selection	Evaluated and selected FastAPI, LangGraph, Playwright, React, Tailwind CSS, Vite, and SQLite.	Nov 2025
3	System Design	Defined functional specifications and drafted UML diagrams (Use Case, Class, Sequence, ERD).	Dec 2025
4	Automation Layer	Developed the custom Playwright browser wrapper for element metadata collection and visual capture.	Jan 2026
5	Core Agent Logic	Built the five LangGraph state nodes and implemented the observe-reason-act reasoning loop.	Jan 2026
6	Backend API	Implemented REST endpoints and asynchronous WebSocket handlers for real-time progress streaming.	Feb 2026
7	Database Integration	Structured SQLAlchemy relational database models and connected the SQLite persistence engine.	Feb 2026
8	Frontend Dashboard	Built the web-based React dashboard and test-authoring workspace using Vite and Tailwind CSS.	Mar 2026
9	System Integration	Linked the React frontend with the FastAPI backend, verifying real-time synchronization of agent execution runs.	May 2026
10	Thesis & Evaluation	Executed benchmark evaluations, completed the final documentation, and compiled the user manual.	June 2026

Table 1.1: Summary of the Project Roadmap

1.8 Report Organization

This report is organized into seven chapters to guide the reader through the entire engineering lifecycle of TestPilot, starting from initial design requirements to the final experimental evaluations.

Chapter 1 introduces the background and motivation of the project, defines the target problem of web test script fragility, outlines the objectives and proposed solution, establishes the scope, and reviews the methodology, technology stack, and timeline.

Chapter 2 provides a detailed review of related work to contextualize the research gap that TestPilot addresses. It traces the historical progression of scripted test automation, details the evolution of generalist autonomous web agents, and analyzes the structural differences of existing autonomous testing baselines.

Chapter 3 presents the system analysis, converting user and stakeholder needs into formal engineering requirements. This chapter documents the stakeholder profiles, details the functional and non-functional specifications, models the system using detailed use case scenarios, and presents a technical and economic feasibility study.

Chapter 4 explains the transition from system analysis to architectural modeling. It details the layered architecture, describes the structural relationships within the class diagrams, documents the execution flow using sequence diagrams, and defines the database entity relationships alongside the API and WebSocket communication contracts.

Chapter 5 provides the implementation narrative. This chapter documents the setup environment, the programming details of the LangGraph state machine node configurations, the SoM visual prompting script, the structured Pydantic tool definitions, the recovery logic, and frontend event handling.

Chapter 6 contains the testing and evaluation of TestPilot, presenting the empirical proof of the design's effectiveness. It documents our testing strategy, explains the experimental setup on the benchmark, and provides a thorough qualitative and quantitative analysis of the results.

Finally, Chapter 7 concludes the report by presenting the engineering conclusions, summarizing the key achievements of TestPilot, documenting lessons learned during development, and proposing future research enhancements.

Chapter 2: Related Work

This chapter presents the theoretical foundations and existing research lines that inform the development of TestPilot. It is divided into three distinct domains: traditional scripted test automation and the structural causes of selector fragility, the evolution of general-purpose autonomous web agents, and the emergence of autonomous software testing agents.

2.1 Overview of Existing Research

Software test automation has moved from static, code-dependent scripts to intelligent, vision-driven systems. This progression sets up the technical challenges that TestPilot resolves.

2.1.1 Scripted Test Automation and Selector Fragility

Scripted web test automation relies on frameworks like Selenium, Puppeteer, and Playwright to drive browsers and execute interactions [10, 16]. A test script runs a hardcoded sequence of operations, querying element identifiers from the active DOM through XPath, CSS selectors, or attributes like id, name, or class.

The core limitation is locator aging [13, 17]. CSS and XPath selectors capture the DOM's structural hierarchy, so any layout or markup change can break the selector path, wrapping a text input in a helper container, or changing a CSS class prefix, is enough. The element's function and appearance haven't changed for the user, but the test fails anyway, and developers are left repairing selectors by hand as the application grows [8, 16].

Researchers have tried to automate these repairs. Early work used DOM structural analysis: when a selector fails, tree-matching algorithms scan the updated DOM for the node most structurally similar to the original [3, 6, 12]. Later work introduced visual-based repair, matching page components through image processing instead of raw DOM nodes [22]. Other approaches synthesize more resilient CSS selectors during recording sessions [17, 18, 21], and self-healing record-and-replay systems update selector definitions dynamically when they detect minor

structural drift [20]. All of these remain reactive as they cannot prevent the initial failure, and they struggle against the structural complexity of modern dynamic SPAs.

2.1.2 Autonomous Web Agents

LLMs have enabled autonomous web agents that execute tasks on web pages without pre-defined scripts [23]. Early agents passed the raw DOM source of the active viewport straight to the language model [11]. But the raw DOM of a modern web app often exceeds 10,000 tokens per page thus noisy, slow to process, and prone to blowing past context limits.

This pushed researchers toward filtering and simplification. Mind2Act uses a small model to rank and filter DOM elements before handing a subset to a larger model like GPT-4 for action prediction [7]. WebAgent takes a similar route with HTML-T5, a sequence-to-sequence model that extracts relevant HTML snippets and plans, while a separate model generates the browser interaction scripts [11]. AutoWebGLM instead combines OCR on screenshots with simplified HTML trees to build a compact observation space [15].

Multimodal LLMs shifted the field toward vision. Agents can now read the visual interface directly, resolving ambiguities that invisible or overlapping DOM elements create. WebVoyager combines screenshots with filtered text to plan and act [14]. CogAgent, an 18-billion parameter model built for high-resolution document and GUI perception, skips DOM parsing entirely and outputs raw pixel coordinates [14]. UI-TARS goes further: a native 72-billion parameter GUI agent that predicts relative coordinate actions straight from screen observations [27].

Bridging visual intent and browser execution requires grounding. Set-of-Mark (SoM) prompting simplifies this by overlaying numbered, colored bounding boxes on interactive elements before the screenshot is captured [24].

2.1.3 Autonomous Test Agents

Autonomous web agents are judged on open-ended task completion, like buying an item. Autonomous test agents face a harder constraint: strict step ordering and explicit validation rules. Feldt et al. defined the analytical framework for Autonomous Test Agents (ATAs), introducing

conversational agents that interpret natural language instructions and act as testing assistants they showed that language models could drive exploratory testing and bug reproduction by simulating human interaction [9].

Early ATAs targeted exploratory mobile testing. GPTDroid models mobile GUI testing as a continuous question-and-answering task: it extracts the page's static and dynamic context, encodes it into a prompt, and queries an LLM for the next human-like action [19]. It achieves high activity coverage, but it's an exploratory tool, it can't execute pre-defined, sequential test scripts or check assertions a human tester wrote.

Chevrot et al. moved the field from exploration to structured test execution, building a benchmark of 113 professionally written end-to-end test cases across three offline applications to evaluate agents under strict validation constraints [1]. They introduced two open-source ATA baselines: SeeAct-ATA, which adapts the SeeAct agent [25] by encoding test progression and validation instructions into a single prompt, and PinATA, a multi-agent design. PinATA reached roughly 60% verdict accuracy and 94% sensitivity, but both baselines produced a high rate of false failure reports [1], the gap TestPilot's structured state tracking and error recovery are built to close.

2.2 Comparative Analysis

To identify the structural and functional innovations of TestPilot, it is necessary to compare its architecture with existing academic baselines and commercial automated web testing platforms. This comparative analysis focuses on four dimensions: input format flexibility, step orchestration models, page observation methods, and error recovery mechanisms.

The primary systems evaluated in this comparison include SeeAct-ATA [1] and PinATA [1] (the established academic baselines), and three commercial AI-assisted web testing platforms: Applitools [3], Functionize [4], and Testim [5].

Table 2.1 summarizes the architectural and operational differences between TestPilot and these related systems.

Feature	SeeAct-ATA	PinATA	Applitools	Functionize	Testim	TestPilot
Input Format	Natural Language test case	Natural Language test case	Natural Language or Recorded	Natural Language or Recorded	Recorded or Coded script	Natural Language test case
Orchestration Model	Single Prompt	Multi-Agent Orchestrator	Autonomous Runner	Workflow Engine	Sequential Player	LangGraph State Machine
Intermediate Assertions	Inferred by LLM	Dedicated Assertor	Visual and Natural Language	Dedicated Assertor	Dedicated Assertor	Dedicated Assertor
Observation Method	Screenshot and DOM choices	Screenshot and SoM	Visual AI and DOM	Screenshot and DOM	DOM and Visual	Screenshot and SoM
Recovery Logic	None	Orchestrator Retry	Self-healing	Self-healing	Self-healing	Orchestrator Retry
Step-Level Trace	No	Yes	Yes	Yes	Yes	Yes
Licensing Model	Open-source	Open-source	Proprietary	Proprietary	Proprietary	Open-source
Evaluation Benchmark	ATA Benchmark	ATA Benchmark	Proprietary Suites	Proprietary Suites	Proprietary Suites	ATA Benchmark

Table 2.1: Comparison of TestPilot with related autonomous test agents and commercial AI-assisted testing platforms.

2.2.1 Input Formats and Step Orchestration

Commercial platforms like Applitools, Functionize, and Testim require recorded interactions, programmatic scripts, or proprietary natural language formats. Because these platforms generate execution paths during recording, they depend on the active application version at that moment. Conversely, SeeAct-ATA, PinATA, and TestPilot accept unstructured natural language test cases directly from project requirements, which bypasses manual recording and scripting.

Orchestration mechanisms also differ. SeeAct-ATA uses a single-prompt model to track steps, select elements, and run assertions. This single reasoning thread often loses execution state, which leads to skipped steps or premature termination. PinATA resolves this by dividing execution among three independent agents: an orchestrator, an actor, and an assertor. TestPilot formalizes this multi-agent structure into a LangGraph state machine with defined nodes and conditional transitions. This stateful directed graph enforces sequential step execution and prevents the state drift common in single-prompt models.

2.2.2 Assertion Verification and Observability

To observe pages, SeeAct-ATA combines screenshots with DOM-extracted text, which scales poorly on large pages. Proprietary systems like Applitools use closed-source visual algorithms and DOM inspection. PinATA and TestPilot use Set-of-Mark (SoM) visual prompting. Specifically, TestPilot injects a JavaScript module to identify elements in the accessibility tree and draw numbered boundaries on the screen. This approach reduces token overhead and improves element identification.

For validation, SeeAct-ATA infers step success implicitly, which causes false positives and missed failures. PinATA, Functionize, and Testim evaluate page states using dedicated assertor components. TestPilot integrates this check as an assertor node within its state machine. Before advancing, this node validates the visual and structural page state against the expected natural language outcome. If verification fails, the machine halts execution and records the failure.

2.2.3 Error Recovery and Reproducibility

Commercial platforms handle transient failures through self-healing selectors that find alternative DOM paths, though this approach cannot recover from logical shifts or unexpected page transitions. SeeAct-ATA lacks recovery mechanisms and terminates immediately upon error. PinATA and TestPilot manage errors via retry loops. TestPilot routes failures to a dedicated Recover node, which analyzes the screenshot, target action, expected assertion, and retry count to

classify the issue as recoverable or terminal. If recoverable, the node pauses and retries execution; bounded retries prevent infinite loops.

Additionally, Applitools, Functionize, and Testim are proprietary systems evaluated on private applications, which prevents direct comparison. SeeAct-ATA, PinATA, and TestPilot are open-source and benchmarked on the public WebArena dataset. This open evaluation allows independent replication of TestPilot's execution speed and accuracy.

Chapter 3: System Analysis

This chapter provides the formal system analysis for TestPilot, defining the stakeholder environment, functional requirements, non-functional requirements, use case diagram, and feasibility study. The analysis translates the high-level project objectives into specific, testable, and quantifiable engineering specifications.

3.1 Stakeholder Analysis

A stakeholder analysis ensures that TestPilot satisfies the operational needs of all participating roles in the software development lifecycle.

Stakeholders:

1. Quality Assurance (QA) Engineers: As the primary operators of TestPilot, QA engineers author natural language test cases, configure environments, run tests, and evaluate failures. They expect reliable execution without manual locator updates. On test failure, they require structured step-by-step traces, historical execution logs, and annotated screenshots to distinguish target application defects from agent grounding errors.

2. Software Developers: Developers use TestPilot execution reports to reproduce and fix defects. They expect the system to tolerate minor HTML changes without triggering false failure alerts, which prevents wasted effort on fragile script maintenance. When failures occur, they require low-level debug data, including the exact step, the attempted action, and the DOM state at the time of failure.

3. Product Managers and Business Analysts: These stakeholders define product requirements and acceptance criteria. They expect to write and review test scenarios in plain text mapped directly to requirements, without writing code or configuring browser selectors. TestPilot allows them to verify functional completeness directly against specifications, which accelerates release validation.

3.2 Functional Requirements

The functional requirements define the active capabilities, inputs, processing steps, and outputs that TestPilot must support.

3.2.1 Test and Project Management

- FR-1: Project Lifecycle Management

  The system must support the complete lifecycle of testing projects. Users must be able to create, view, edit, and delete individual projects. Each project acts as an isolated logical container that groups related test cases and stores system-level metadata, such as the starting base URL. When a project is created, the system must generate a unique Universally Unique Identifier (UUID) and persist the record to the relational database. The backend must expose these CRUD operations through REST API endpoints mounted under the /api/projects prefix.

- FR-2: Test Case Management

  Within each project container, the system must support the complete lifecycle of test cases. Users must be able to create, view, update, and delete individual test cases. Each test case is defined by a unique name, a specific starting URL (target_url), an optional external reference string (external_id), and a binary test type (type) representing positive (P) or negative (N) execution logic. The backend must persist these configurations in the relational database, establishing foreign key relationships to their parent projects.

- FR-3: Step-By-Step Test Definition

  The system must store each test case as an ordered sequence of natural language steps. Each step must contain an action instruction string (such as "enter the password into the field") and may include an optional expected result assertion string (such as "the dashboard is displayed"). The system must preserve the sequential order of steps by storing them as a serialized JSON array (steps_json) directly within the corresponding test case record. The

step order is enforced implicitly by the indices of the JSON array, representing the step numbers.

- FR-4: Test Step Manipulation

  The user interface must enable users to modify the step sequence within a test case. Users must be able to add new steps at any position, edit existing text instructions, delete obsolete steps, duplicate actions, and dynamically reorder steps within the execution array. The backend must modify and re-serialize the JSON array elements in the steps_json column when steps are inserted, deleted, or reordered, which prevents sequence gaps without requiring row-recalculation overhead.

3.2.2 Automated Execution and Orchestration

- FR-5: Selector-Free Agent Execution

  TestPilot must execute natural language test cases in a live browser session without requiring pre-written CSS selectors or XPath locators for the interactive elements. The system must launch a Chromium browser instance via Playwright, inject the SoM element-labeling script, and coordinate actions via a LangGraph state machine. This state machine dynamically identifies element coordinates at runtime by passing annotated screenshots and filtered accessibility tree structural metadata to a multimodal Large Language Model, which predicts the coordinates of the selected elements on the active viewport.

- FR-6: Step-Level Visual Artifact Capture

  During test execution, the system must capture and record visual artifacts to support debugging and trace verification. The browser controller must capture at least one high-resolution page screenshot for every executed step in the sequence. If a step fails due to an execution error or an assertion failure, the system must capture additional screenshots showing the exact visual state of the page at the point of failure. The system must capture these screenshots as high-resolution JPEG/PNG files and persist them directly to the

database as binary large objects (BLOB data) in the screenshots table, categorized by kind (observation or verification).

- FR-7: Automatic Verdict and Failure Diagnosis

  Upon completing or halting a test execution run, the system must generate a binary verdict (Passed or Failed) and record this output in the database. If the test run fails, the system must record the exact step index where the failure occurred. It must also record a detailed diagnostic string explaining whether the failure was caused by an action error (such as an element not being interactable after multiple retries) or an assertion failure (where the visual state did not match the expected outcome). These run outcomes, along with the counts of passed and failed steps, must be written to the test_runs and step_results tables.

3.2.3 Data Streaming and Historical Reporting

- FR-8: Real-Time State Streaming

  The backend must serialize the execution state of the active test run and stream this data to the web frontend in real time. During execution, the system must establish an active WebSocket connection at /api/test-cases/{id}/execute. After each step transition in the LangGraph state machine, the system must stream serialized JSON event frames (such as step.started, agent.observation, or step.completed) containing the active step index, step execution status, base64-encoded visual observations, and raw logging entries.

- FR-9: Historical Execution Logging

  The system must persist the metadata of every completed test execution in the database. Each test run record must store the execution status (Passed, Failed, Error, Aborted), the execution duration (duration_seconds), the total steps passed (steps_passed), the total steps failed (steps_failed), and the specific Large Language Model and provider used. Timestamps for both run initiation (started_at) and run completion (completed_at) must

also be recorded. Additionally, execution log lines must be captured and persisted as RunLogEntry rows in the database rather than filesystem logs.

- FR-10: Step-Level Historical Inspection

  The system must enable detailed historical analysis of completed test runs. The user interface must allow users to retrieve and inspect any past test execution run at the step level. To support this, the database must query the step_results table, executing joins with the screenshots table via screenshot_observation_id and screenshot_verification_id foreign keys. This query retrieves and displays the exact text instruction, the expected assertion, the actual action performed by the agent, the number of retry attempts required, the step verdict, and the corresponding page screenshot.

3.3 Non-Functional Requirements

Non-Functional Requirements define the operational constraints, quality attributes, performance boundaries, and safety policies that TestPilot must satisfy to be viable for software engineering environments.

3.3.1 Performance and Latency

- NFR-1: Observation Processing Latency

  The system must complete the processes of injecting JavaScript, rendering the SoM visual overlay, capturing the page screenshot, and streaming the visual artifacts to the client over WebSockets within a local processing latency of 500 milliseconds. This threshold excludes external Large Language Model network round-trip latency.

- NFR-2: Database Query Performance
  Database operations executed through the SQLAlchemy ORM must execute in under 50 milliseconds for standard CRUD operations. This limit keeps the main asynchronous event loop unblocked and responsive to incoming API requests.

3.3.2 Security and Data Protection

- NFR-3: Database Transaction Security

  All database interactions must use parameterized queries through the SQLAlchemy ORM to prevent SQL injection vulnerabilities. Direct execution of raw, unparameterized SQL strings is prohibited.

- NFR-4: Execution Context Isolation

  The browser controller must launch isolated browser contexts for every test run. The system must guarantee that separate runs do not share cookies, active session tokens, local storage cache, or browser history, which prevents cross-test data pollution.

3.3.3 Usability and Human Factors

- NFR-5: Visual Test Case Authoring

  The system must enable complete project setup, test case creation, and step manipulation through a visual interface. No programming knowledge, CSS selector writing, or script generation is required to build or execute tests.

- NFR-6: Interface Responsiveness

  The web client, built on React, Tailwind CSS, and Vite, must render correctly on standard desktop resolutions (1920x1080). Visual run state updates and log streams must update the user interface within 100 milliseconds of being received via WebSockets.

3.3.4 Compatibility and Portability

- NFR-7: Operating System Portability

  The backend and database layers must execute natively or via Docker containerization on Windows 10+, macOS 12+, and Ubuntu 20.04+ systems.

- NFR-8: Browser Driver Compatibility

  The execution layer must support headless and headful Chromium browser instances using the Playwright driver across all target operating systems.

- NFR-9: Web Client Compatibility

  The frontend application must execute without functional degradation on modern web browsers, including Google Chrome, Mozilla Firefox, and Apple Safari.

3.4 Use Case Diagram

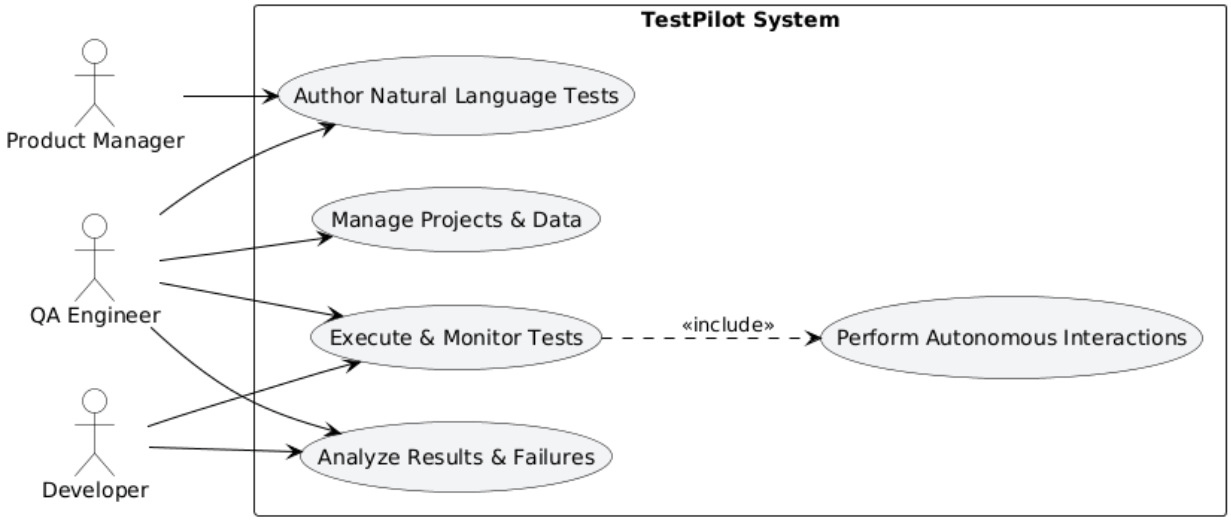


Figure 3.1: Use Case Diagram

The system interactions, actor roles, and functional boundaries of TestPilot are modeled using the Use Case Diagram presented in Figure 3.1. This diagram provides a formal representation of how different stakeholders interact with the application services and defines the logical scope of the system.

3.4.1 Actor Roles and Responsibilities

Three primary human actors reside outside the system boundary and interact with the application: the Product Manager, the QA Engineer, and the Developer. Each actor represents a distinct user role with specific access privileges and operational workflows.

- The Product Manager interacts primarily with the Author Natural Language Tests use case. This association enables non-technical stakeholders to write and define acceptance criteria directly as sequential, plain-text steps without code generation or CSS selector management.
- The QA Engineer is the most active actor in the system and is associated with four use cases: Author Natural Language Tests, Manage Projects & Data, Execute & Monitor Tests, and Analyze Results & Failures. This actor possesses full administrative control to organize test suites, manage project metadata, initiate test execution runs, and review step-level historical data.
- The Developer is associated with the Execute & Monitor Tests and Analyze Results & Failures use cases. This allows developers to trigger validation tests after user interface refactoring and inspect the step-level execution traces and captured screenshots to identify and resolve functional regressions.

3.4.2 Use Case Specifications and the Include Relationship

The system defines five primary use cases inside the boundary block. The Author Natural Language Tests use case parses and saves plain-text instructions into ordered step records in the database. The Manage Projects & Data use case handles project creation, test case grouping, and relational schema mapping. The Analyze Results & Failures use case retrieves historical run metadata, visual screenshots, and step logs from the database for post-run evaluation.

The core execution logic is split into two connected use cases: Execute & Monitor Tests and Perform Autonomous Interactions. The Execute & Monitor Tests use case represents the user-triggered run sequence. It is associated with both the QA Engineer and the Developer, who initiate test runs and monitor the live visual states, step statuses, and execution logs via WebSocket streams.

The dependency from Execute & Monitor Tests to Perform Autonomous Interactions is modeled as an «include» relationship. In UML specifications, an include relationship represents a mandatory dependency where the base use case cannot complete without invoking the included use case.

In TestPilot, triggering a test execution run automatically initiates the browser execution loop. The system launches a headless Chromium instance, injects the element-labeling JavaScript, renders the SoM visual overlay, and processes visual observations to execute actions and verify assertions. This autonomous loop executes entirely under the hood without manual intervention or separate actor initiation, which is correctly captured by the included relationship.

3.5 Feasibility Study

This assesses whether TestPilot can be developed, operated, and maintained within standard software engineering environments.

1. Technical Feasibility

   The system relies on mature, open-source libraries. The backend uses Python, FastAPI, and SQLAlchemy, which are stable technologies with established documentation. For agent control, LangGraph provides a state machine model that prevents erratic execution transitions. Playwright handles browser automation and Chromium interaction directly.

   Cloud-based multimodal LLM APIs present the primary technical risks, specifically regarding network latency and API rate limits. TestPilot addresses these limitations through its Recover state node, which executes local retry mechanisms and error classification. Based on this architecture, the technical requirements of the project are achievable.

2. Economic Feasibility

   The open-source stack (including FastAPI, React, Playwright, and SQLite) eliminates upfront software licensing expenses. Consequently, development costs are limited to engineering hours. Operational expenses consist of API token consumption, which scales with the size and frequency of test runs. This variable cost structure remains low compared to the licensing fees of commercial automated testing platforms.

3. Operational Feasibility

TestPilot integrates into daily software engineering workflows without requiring programming skills, locator maintenance, or compiler configuration. QA engineers and product managers can execute tests directly using the natural language specifications already present in business requirements documents.

A web-based React frontend provides a dashboard to manage projects and monitor active execution states via WebSockets. By displaying step-by-step visual failure traces, the system simplifies debugging. This design fits directly into existing agile development pipelines.

Chapter 4: System Design

This chapter describes the system design of TestPilot and translates functional requirements into an implementable architecture. It details the multi-layered system structure alongside the class and sequence diagrams that coordinate execution and recovery. Finally, the chapter outlines the database schema, user interface wireframes, and security designs for session isolation and key containment.

4.1 System Architecture

TestPilot uses a decoupled, layered architecture to isolate system concerns. These divisions comprise the presentation, API and service, orchestration, execution and automation, and persistence layers. Figure 4.1 shows the structural layout of these layers and the flow of data, instructions, and events across system boundaries.

4.1.1 Overview of the Architecture

The architecture enforces unidirectional downward dependencies, which isolates client-side user interactions from backend database transactions. A network boundary separates the client-side presentation layer from the server-side backend. Two communication protocols traverse this boundary: stateless HTTP REST requests for configuration management, and persistent, bidirectional WebSocket connections for real-time execution streaming.

The backend separates data persistence and agent execution into parallel, independent pipelines. This separation ensures that database-write latency, particularly during the storage of large screenshots, does not block the browser automation loop or delay model reasoning.

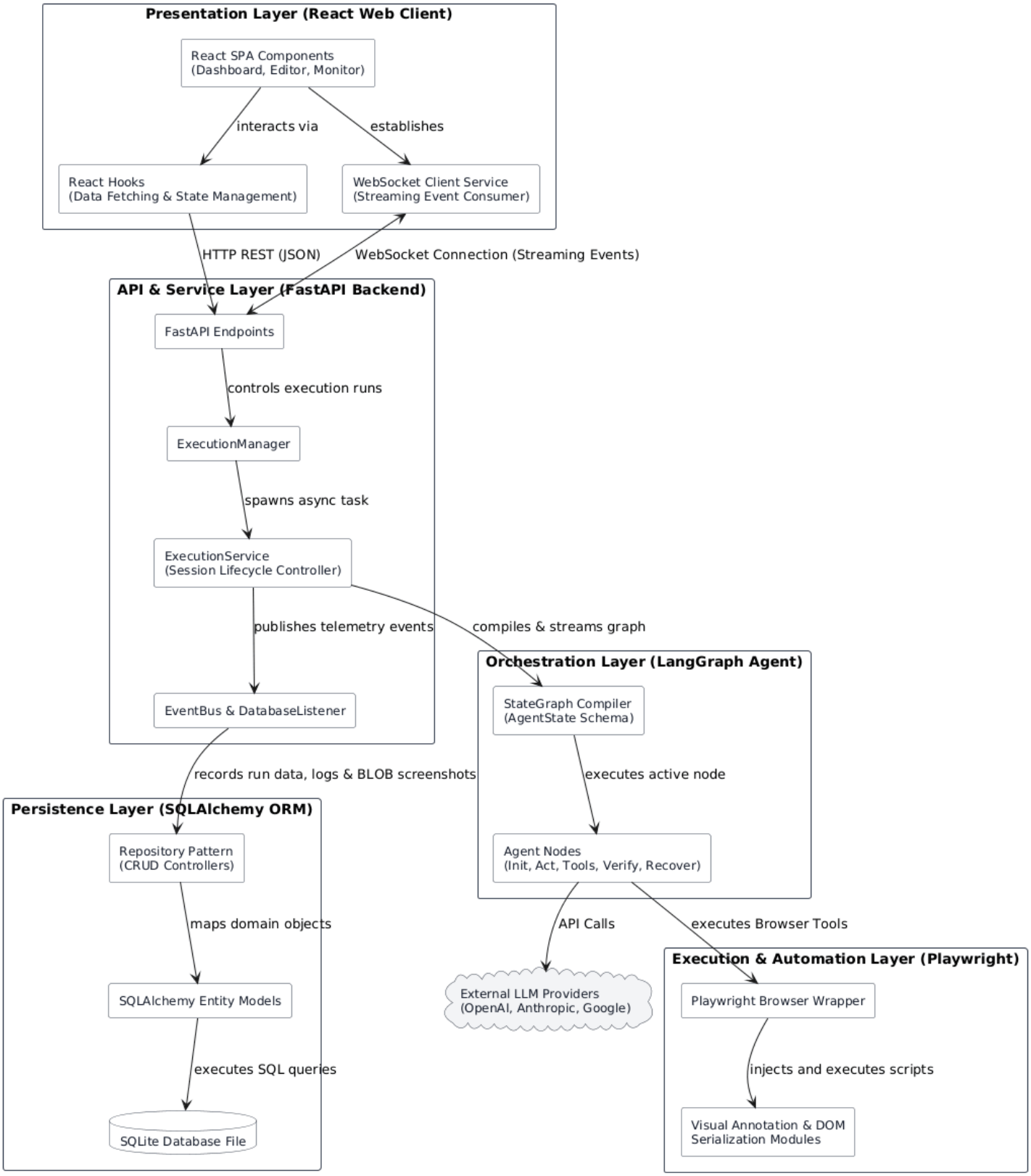


Figure 4.1: System Architecture Diagram

4.1.2 Presentation Layer

The presentation layer contains the client-side user interface of TestPilot, which is developed as a single-page web application (SPA). The build and compilation pipeline is managed by Vite 8. The user interface is built on React 18, utilizing concurrent rendering to ensure responsive visual states.

Client-side routing and path translation are handled by React Router DOM 6. This library manages the active views, including the project list, the interactive test case authoring workspace, and the historical execution reports.

To decouple user interface components from server-state synchronization and caching complexities, the client uses TanStack React Query 5. All data retrieval and mutation operations are managed through this library, which organizes server data into structured cache keys. This design ensures that the web client only initiates network requests when the underlying server state is marked as stale.

All client-to-server communications are managed by two modules:

- HTTP Client: A thin wrapper built around the native web fetch API. It automatically structures request headers, handles JSON serialization, and translates non-2xx HTTP responses into structured client exceptions.
- WebSocket Client Service: A custom React state hook that manages the full lifecycle of the streaming execution connection. This service opens the WebSocket channel, transmits the initial run configuration parameters, consumes incoming JSON telemetry frames, and triggers cache invalidation once a terminal execution frame is received.

The visual interface is styled using Tailwind CSS 3 utility classes, which avoids external CSS compilation dependencies and maintains visual consistency across different browser resolutions.

3.1.3 API and Service Layer

The API and service layer acts as the entry gateway to the backend application, running on FastAPI. This layer is responsible for receiving client requests, validating inputs, managing execution, and publishing runtime events.

The entry point of this layer is the FastAPI router suite, which mounts three distinct endpoint modules: projects, test cases, and test runs. These routers parse incoming REST payloads, execute basic authorization, and delegate data operations to the persistence layer.

The core of the backend execution logic is managed by two service classes:

- ExecutionManager: This class serves as the gate for the application. It maintains an active execution dictionary protected by an asynchronous mutual exclusion lock. To prevent data corruption, the manager restricts execution to a single concurrent run per test case. If a user attempts to execute an already active test case, the manager rejects the request and throws a conflict exception.
- ExecutionService: This class controls the lifecycle of an active test run. It compiles the LangGraph state machine, initializes the starting state variables, launches the asynchronous execution task, and processes final execution verdicts.

To decouple test execution from data persistence, the service layer implements an asynchronous pub/sub model using an EventBus and a DatabaseListener. As the execution task progresses, the system publishes telemetry events to the EventBus. The DatabaseListener subscribes to these events and persists them to the database in the background. This design ensures that database transaction latency does not block the real-time reasoning loop of the agent.

4.1.4 Orchestration Layer

The orchestration layer contains the decision-making logic of the agent, implemented as a stateful graph using LangGraph. The graph coordinates the execution state across node boundaries using a central state dictionary named AgentState. This state structure tracks the conversation history,

the active step index, step actions, expected assertions, retry counts, visual screenshot bytes, and a global key-value memory store. State variables are preserved across node boundaries using LangGraph reducer functions, which accumulate values across node invocations while preventing data loss.

The state transitions and logical execution flow of the orchestration layer are visualized in Figure 4.2.

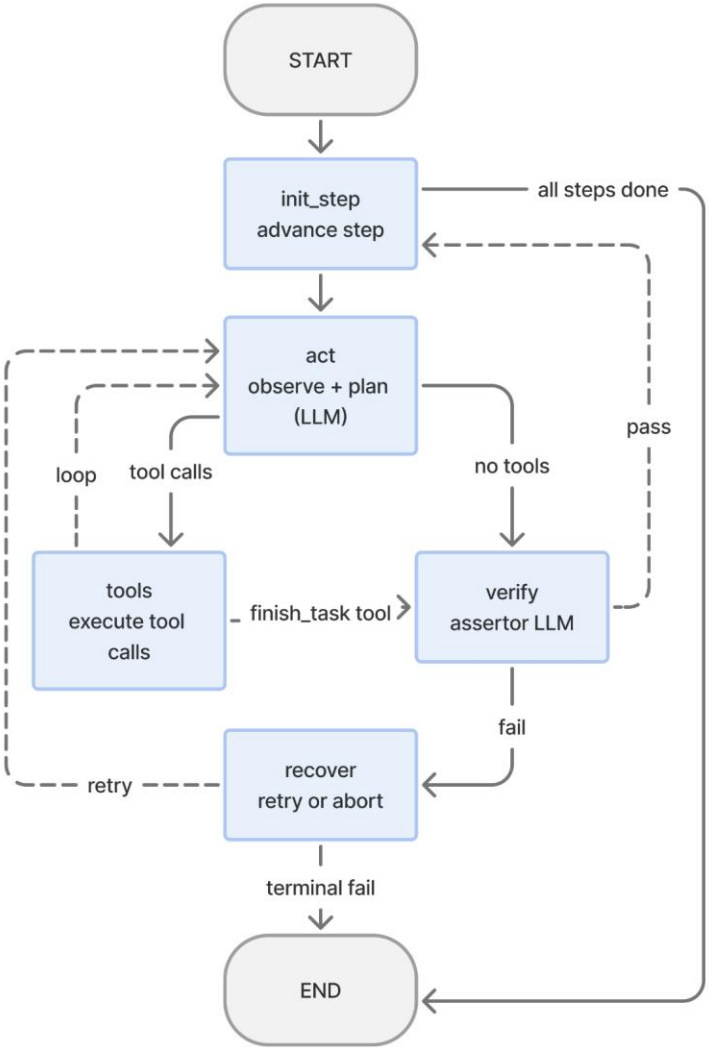


Figure 4.2: Agent Architecture and execution flow

As shown in the agent architecture diagram, the state machine registers five specialized nodes and manages execution through three primary loops:

- The Step Initialization and Completion Path: The execution begins at START and enters the init_step node, which advances the step index and prepares the step parameters. If the system detects that all steps have been executed successfully (all steps done), the graph routes directly from init_step to END. Otherwise, the execution transitions to the act node.
- The Actor-Tool Internal Reasoning Loop: Once the act node is invoked, the agent captures the visual state, plans its next action, and chooses whether to execute a browser tool. If the model generates tool calls, the state machine transitions to the tools node to execute the browser operations via Playwright. The graph then routes back to the act node to re-observe the page. This loop repeats until either the model predicts no further tool calls are required (no tools) or the model invokes the explicit finish_task tool, both of which route the execution directly to the verify node.
- The Verification and Recovery Fail-Safe Loop: The verify node removes visual overlays, captures a clean screenshot, and evaluates the visual and structural page state. If the assertion passes (pass), the graph routes back to the init_step node to advance the step index. If the assertion fails (fail), the graph transitions to the Recover node. The Recover node evaluates the failure context and decides whether to trigger a step retry (which resets the loop counter and routes back to the act node) or transition to END on a terminal fail, which halts the execution immediately.

To perform visual analysis, action planning, and error recovery, the orchestration layer queries external providers using structured JSON API calls. It uses three distinct model configurations: the actor model (with browser tools bound to its schema), the assertor model (with structured output bound to validation schemas), and the recovery model. This specialized separation of model roles prevents reasoning conflicts and ensures higher execution accuracy.

4.1.5 Execution and Automation Layer

The execution and automation layer provides a dedicated browser abstraction interface, wrapping Playwright's asynchronous API to drive Chromium browser instances. The Playwright Browser Wrapper manages the browser lifecycle, handles page navigation stability using DOM element counters, and captures page screenshots.

To interact with pages without hardcoded selectors, this layer injects two specialized client-side JavaScript modules into the browser context:

- Visual Annotation Module: This module identifies interactive elements in the browser accessibility tree, filters out invisible components, calculates on-screen coordinates, and draws high-contrast visual overlays. This provides a SoM presentation layer directly on the active viewport.
- DOM Serialization Module: This module converts interactive DOM elements into compact, descriptive strings. This minimizes token consumption when elements are passed to the Large Language Model.

The browser driver exposes high-level interaction tools (such as clicking, typing, selecting, and scrolling) to the orchestration layer, translating numerical visual labels into active browser events at runtime.

4.1.6 Persistence Layer

TestPilot uses a consolidated database model for all transactional and binary data, running entirely on SQLite 3. The backend manages database operations using the SQLAlchemy 2.0 Object-Relational Mapper (ORM), implementing the Repository pattern to expose stateless CRUD interfaces to the service layer.

The database schema is divided into three configuration tables (projects, test_cases, test_runs) and three telemetry tables (screenshots, step_results, run_log_entries). To enforce relational integrity, the database engine executes a startup pragma listener (PRAGMA foreign_keys=ON) on every new connection.

All captured page screenshots are saved directly as binary large objects (BLOB data) in the screenshots table. Similarly, execution logs are persisted as discrete rows in the run_log_entries table. Database sessions are managed using an explicit context manager that commits transactions on clean exits, rolls back on exceptions, and always closes the session to prevent database locks during parallel execution runs.

4.2 Class Diagrams

The structural relationships and class-level interactions of TestPilot are documented through three distinct class diagrams. Rather than presenting a single, monolithic diagram that combines database schemas, HTTP routing, and state machines, the static design is divided into dedicated functional views. This separation ensures that the structural dependencies of the system remain easy to analyze and evaluate.

4.2.1 Service and Event Infrastructure

Figure 4.3 illustrates the class design of the service boundary and the event infrastructure, which manages concurrent execution lifecycles, runs asynchronous tasks, and handles real-time telemetry.

The entry point to this layer is the ExecutionManager class. It restricts execution to a single concurrent run per test case to prevent data collisions. The manager tracks active runs by storing _ActiveExecution instances in a private dictionary protected by an asynchronous lock. Each _ActiveExecution instance is a composition of an ExecutionHandle and a DatabaseListener.

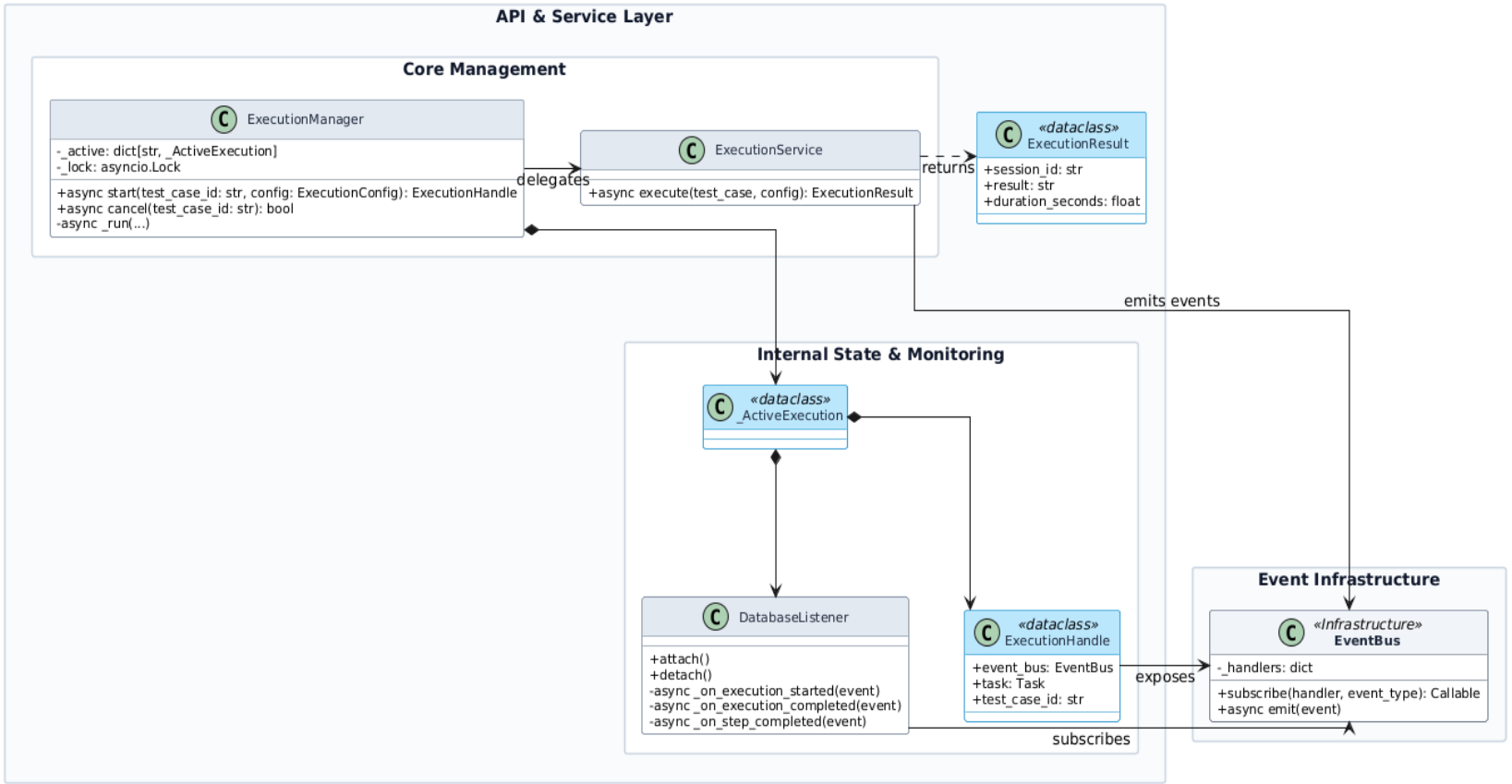


Figure 4.3: Service and Event Infrastructure

The ExecutionHandle encapsulates the asynchronous task and the local EventBus of the run, which provides other modules with access to the execution state. The ExecutionService class is invoked by the manager to execute individual test cases, coordinating the lifecycle of the browser and compiling the state graph. After execution completes, the service returns an ExecutionResult dataclass containing the final run details.

The asynchronous event routing is handled by the EventBus class, which implements an event-driven publish-subscribe model. The DatabaseListener class subscribes to this bus to capture execution milestones, logs, and screenshots, and persists them directly to the database via the repository. This design decouples the active test execution from database transactions, which ensures that write operations do not block the browser automation loop.

4.2.2 Agent Orchestration and Execution

Figure 4.4 deconstructs the classes that manage the decision-making logic and automated browser interactions of TestPilot.

The NodeContext class represents a frozen configuration container injected into every graph node upon construction. It contains shared references to the Browser driver, the session logger, the EventBus, and the three Large Language Model instances (the actor, the assertor, and the recovery models). The orchestration logic is divided among five dedicated node classes: InitStepNode, ActNode, VerifyNode, RecoverNode, and SequentialToolExecutor. Each node class implements an asynchronous __call__ method that accepts the central AgentState as a parameter and returns a state dictionary or a navigation command.

The relationships from the node classes to NodeContext are drawn as solid directed associations. This representation is semantically correct because NodeContext is injected during constructor initialization and stored as a private instance variable within each node, rather than being a temporary method parameter. The shared execution state is defined by the AgentState TypedDict, which tracks the conversation history, active step index, step assertions, and execution results. The Browser class represents the execution driver, wrapping the Playwright API to control Chromium.

It exposes public automation methods, such as goto, click, fill, select, and screenshot, which share a single page session with the NodeContext to execute visual labeling and tool calls.

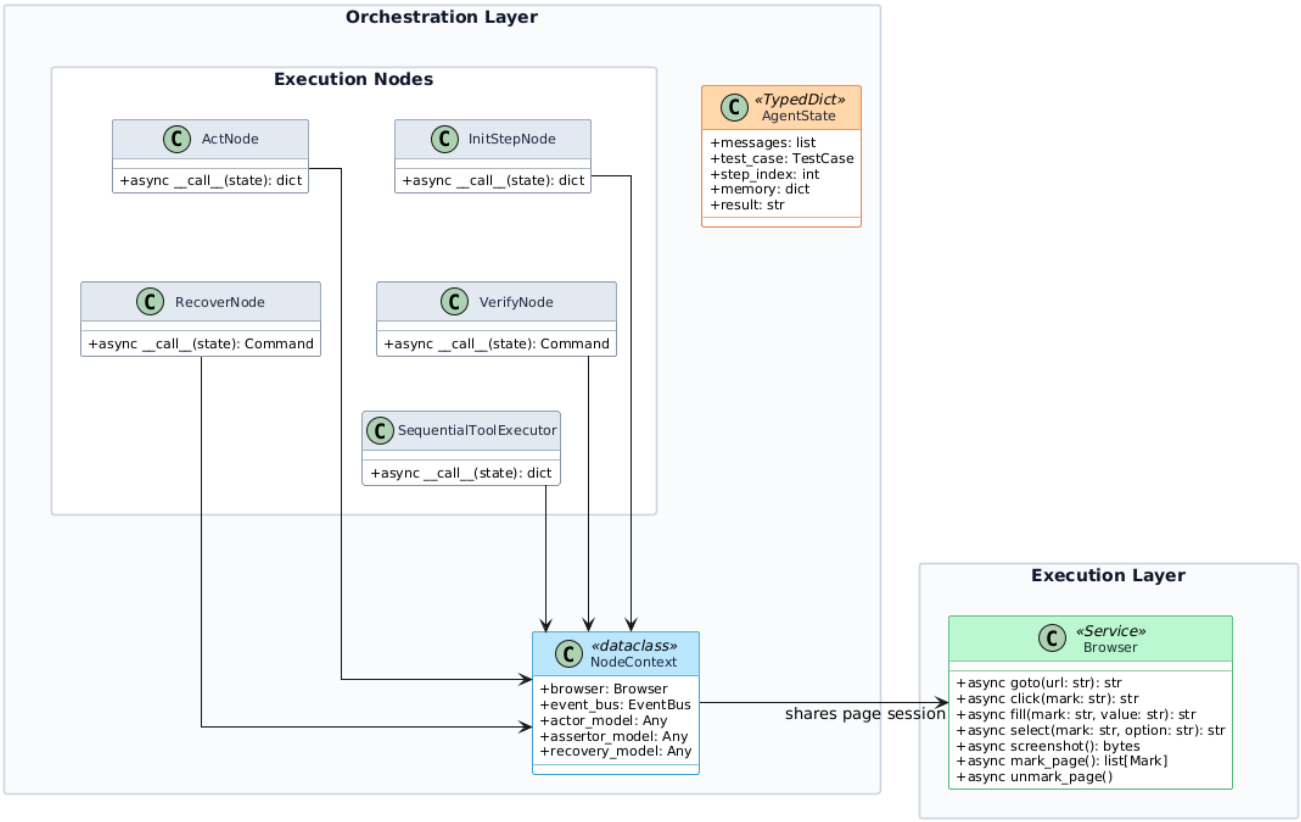


Figure 4.4: Agent Orchestration and Execution

4.2.3 Domain and Persistence Mapping

Figure 4.5 illustrates how TestPilot structures its core business domain objects and represents their corresponding database schemas.

In the Domain Layer, the system defines two lightweight entities: TestCase and TestStep. The TestCase class is structured as a composition of TestStep objects, which preserves the sequential natural language steps written by the user.

In the Persistence Layer, the relational schema is mapped using the SQLAlchemy ORM base class. The relational models include Project, TestCaseORM (labeled TestCase in the SQL schema), TestRun, Screenshot, StepResult, and RunLogEntry. The table structures enforce strict cardinality

constraints: a single Project has zero or many TestCaseORM records; a single TestCaseORM is executed as zero or many TestRun records; and a single TestRun physically owns multiple StepResult, Screenshot, and RunLogEntry records using strong composition.

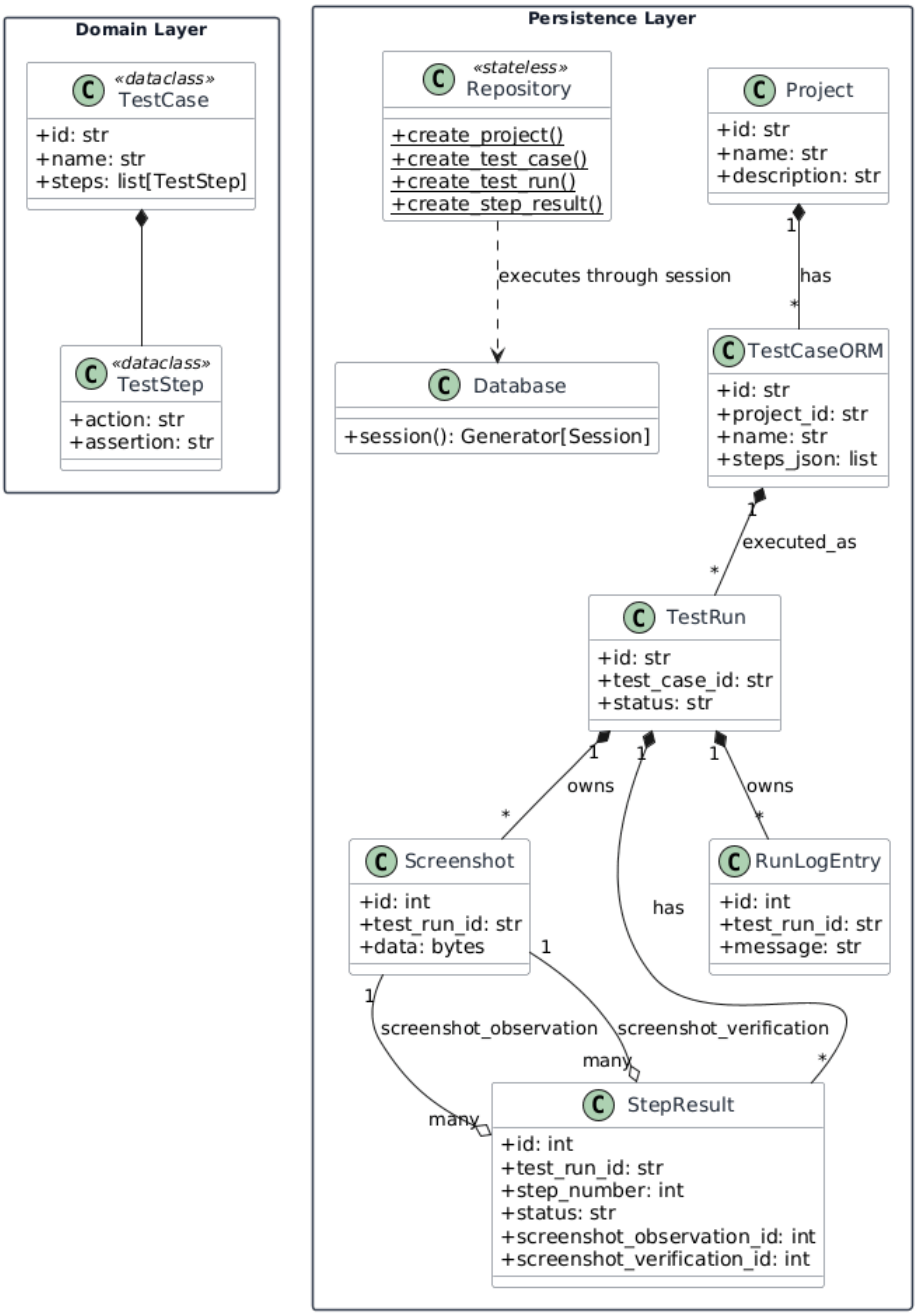


Figure 4.5: Domain and Persistence Mapping

The foreign key references from StepResult to Screenshot are modeled as solid directed associations (screenshot_observation and screenshot_verification) pointing from StepResult to

Screenshot. This representation is semantically correct because Screenshot records are physically owned and deleted by TestRun via composition. Using a directed association instead of an aggregation or a secondary composition respects the single-ownership constraint and models the navigation pathway of the database foreign keys. Database queries and updates are handled by the stateless Repository class, which uses static methods to run parameterized SQL queries using the session context manager provided by the Database class.

4.3 Sequence Diagrams

The dynamic, runtime interactions and asynchronous workflows of TestPilot are documented through three detailed sequence diagrams. These diagrams trace the sequence of operations, message passing, and state changes executed across the system layers during critical use cases.

4.3.1 Test Execution Initialization and Session Establishment

Figure 4.6 illustrates the sequence of operations triggered when a user executes a test case, tracing the handshake from the React frontend to the initialization of the state graph and the database persistence listeners.

The lifecycle begins when a user clicks the execution trigger on the React client. The client initiates a bidirectional WebSocket connection to the FastAPI endpoint at /api/test-cases/{id}/execute. To prevent execution race conditions, the ExecutionManager acquires an asynchronous mutual exclusion lock (asyncio.Lock) before evaluating the request. Under this lock, the manager queries the SQLite database through the Repository to load the target test case configuration and verify that the test case is not already running.

Once validated, the manager instantiates a run-specific EventBus and a DatabaseListener, which attaches itself to the bus to capture execution telemetry in the background. The ExecutionManager then instantiates the ExecutionService and returns an ExecutionHandle containing the active task reference to the FastAPI endpoint, which releases the lock.

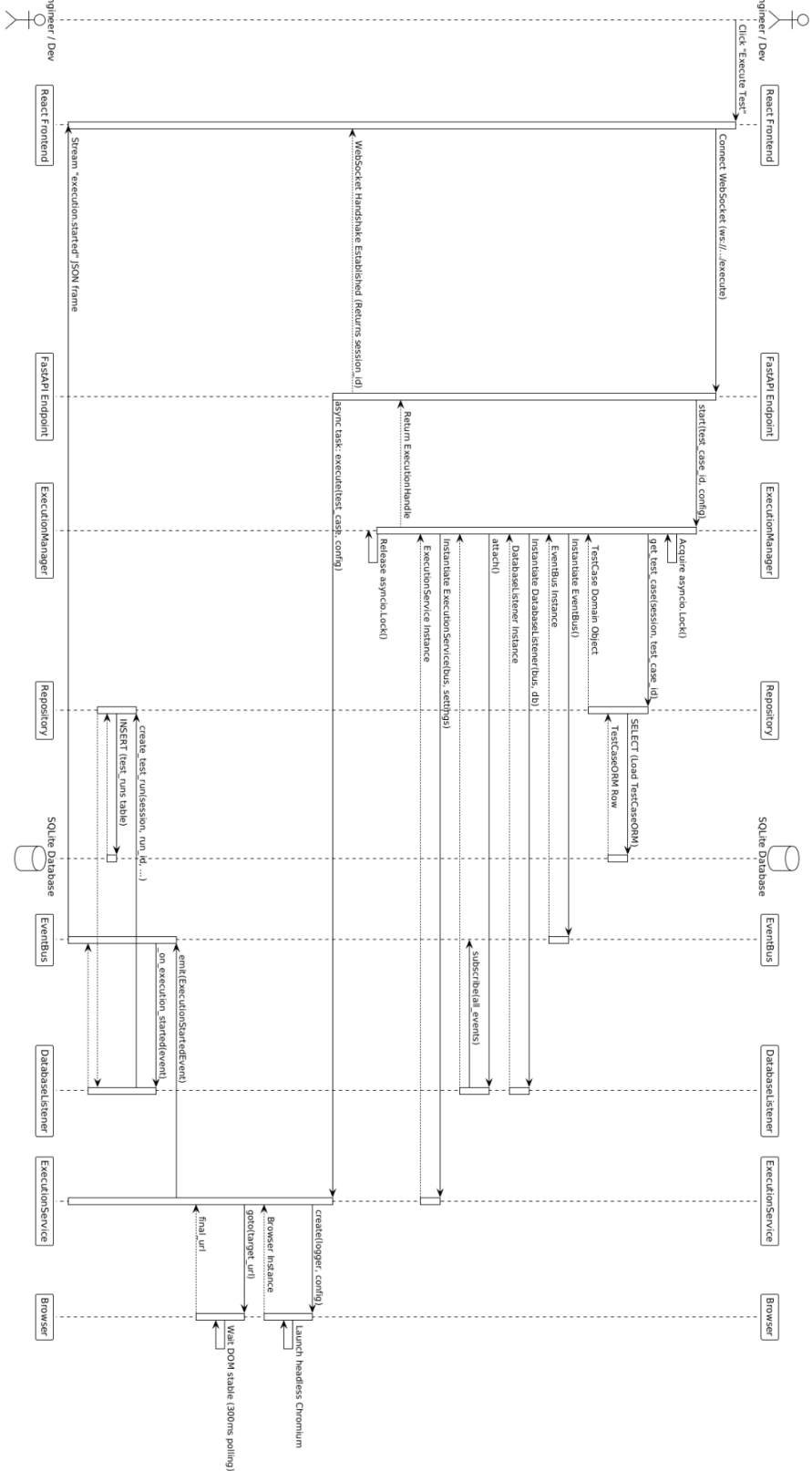


Figure 4.6: Test Execution Initialization and Session Establishment Sequence Diagram

The FastAPI endpoint completes the WebSocket handshake with the client and immediately spawns the ExecutionService.execute() method as a non-blocking asynchronous task. The service launches a headless Chromium instance using Playwright, navigates to the target URL, and waits until the DOM remains stable for 300 milliseconds.

Once stable, the service emits an ExecutionStartedEvent to the EventBus. The DatabaseListener intercepts this event, writes a new record with a status of "running" to the test_runs database table, and begins persisting the execution logs. At the same time, the FastAPI WebSocket handler captures the event from the EventBus and streams the initial execution.started JSON frame back to the client over the active WebSocket channel.

4.3.2 The Core Agent Planning and Tool Execution Loop

Figure 4.7 illustrates the sequence of operations executed during the planning and action phase of a single test step. It traces the interactions between the LangGraph state machine, the visual page annotation scripts, the multimodal language model, and the browser automation driver.

When the AgentGraph invokes the ActNode, the planning phase begins. The node commands the Browser wrapper to execute the markPage() JavaScript function inside the active browser context. The script filters the DOM accessibility tree to identify visible, interactive elements, assigns numerical indexes to them, and renders high-contrast, dashed bounding boxes over the elements on the page viewport.

The Browser then captures a screenshot of this annotated page, compresses it to a lightweight JPEG format, and returns the binary image data to the ActNode. The ActNode emits an ObservationEvent to the EventBus. The FastAPI WebSocket handler intercepts this event and streams the agent.observation JSON frame containing the base64-encoded screenshot back to the client over the active WebSocket channel.

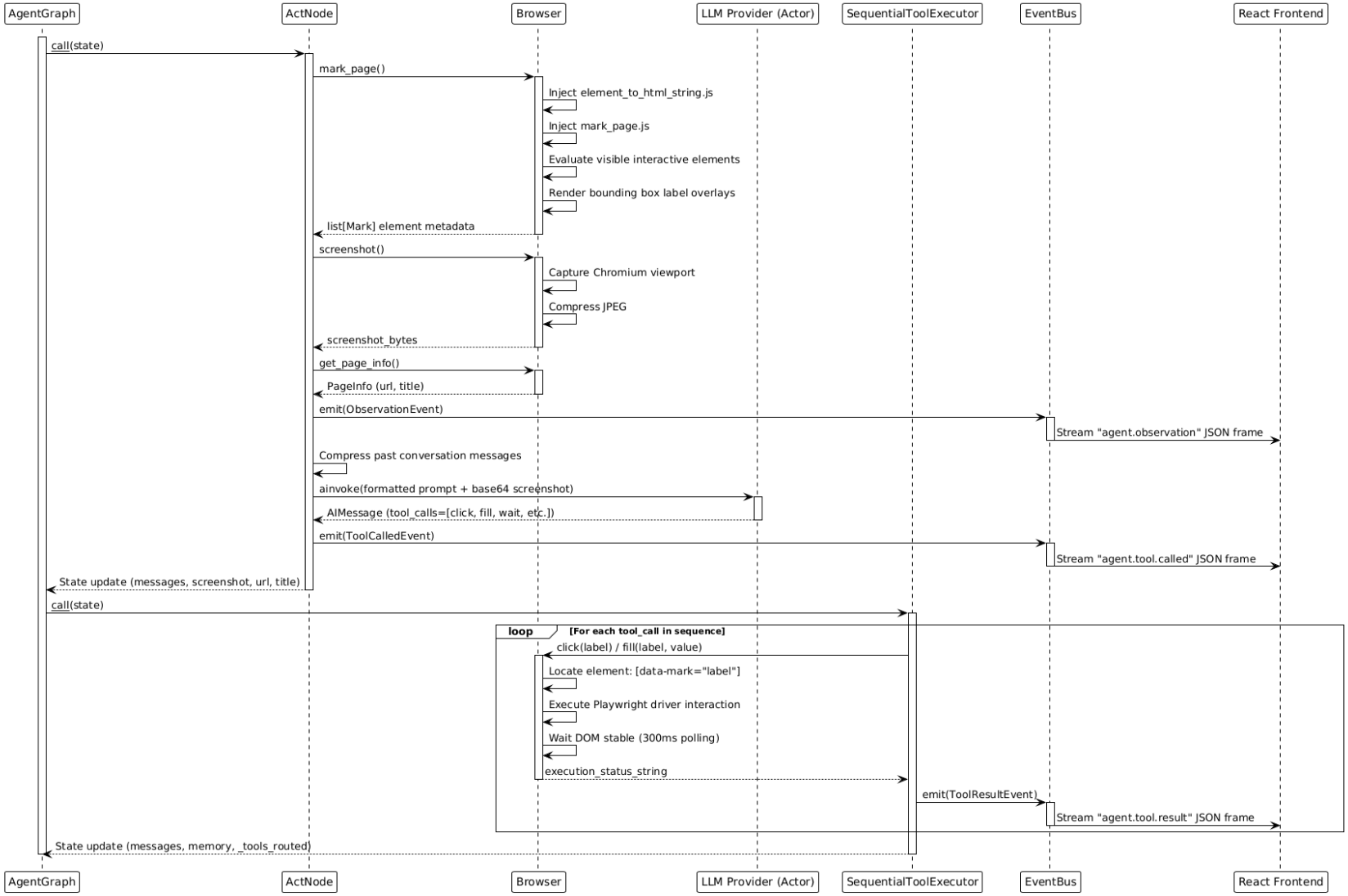


Figure 4.7: The Core Agent Planning and Tool Execution Loop Sequence Diagram

To optimize context window consumption, the ActNode compresses previous multimodal observations into text-only representations before invoking the actor model. It constructs a multimodal message containing the active action instructions, the global memory state, the serialized interactive elements list, and the base64-encoded annotated screenshot. The actor model processes this prompt and returns an AIMessage containing structured tool calls.

The ActNode emits a ToolCalledEvent to the EventBus to notify the client, and returns the tool list to the AgentGraph, which routes the execution directly to the SequentialToolExecutor. The tool executor iterates over the tool calls, locates the target element on the page using its custom data-mark attribute, and executes the interaction using the Playwright API. After each interaction, the system waits for the page to stabilize and publishes a ToolResultEvent to stream the status to the frontend.

4.3.3 Step Verification, Failure Analysis, and Self-Healing Recovery

Figure 4.8 illustrates the sequence of operations executed when a test step transitions to the verification and recovery phases, showcasing how the system evaluates assertions and executes self-healing retries.

Once the action loop terminates, the AgentGraph routes execution to the VerifyNode. To evaluate the page state objectively, the VerifyNode commands the Browser to execute unmarkPage(), which removes all visual bounding boxes from the viewport.

The node then triggers an asynchronous sleep of 500 milliseconds to allow CSS animations, dropdown transitions, and page renders to complete. After this buffer, the Browser captures a clean page screenshot, which is passed to the assertor model alongside the expected natural language assertion. The assertor model uses structured output parsing to return an AssertionResult schema containing a binary passed boolean and a detailed reason string.

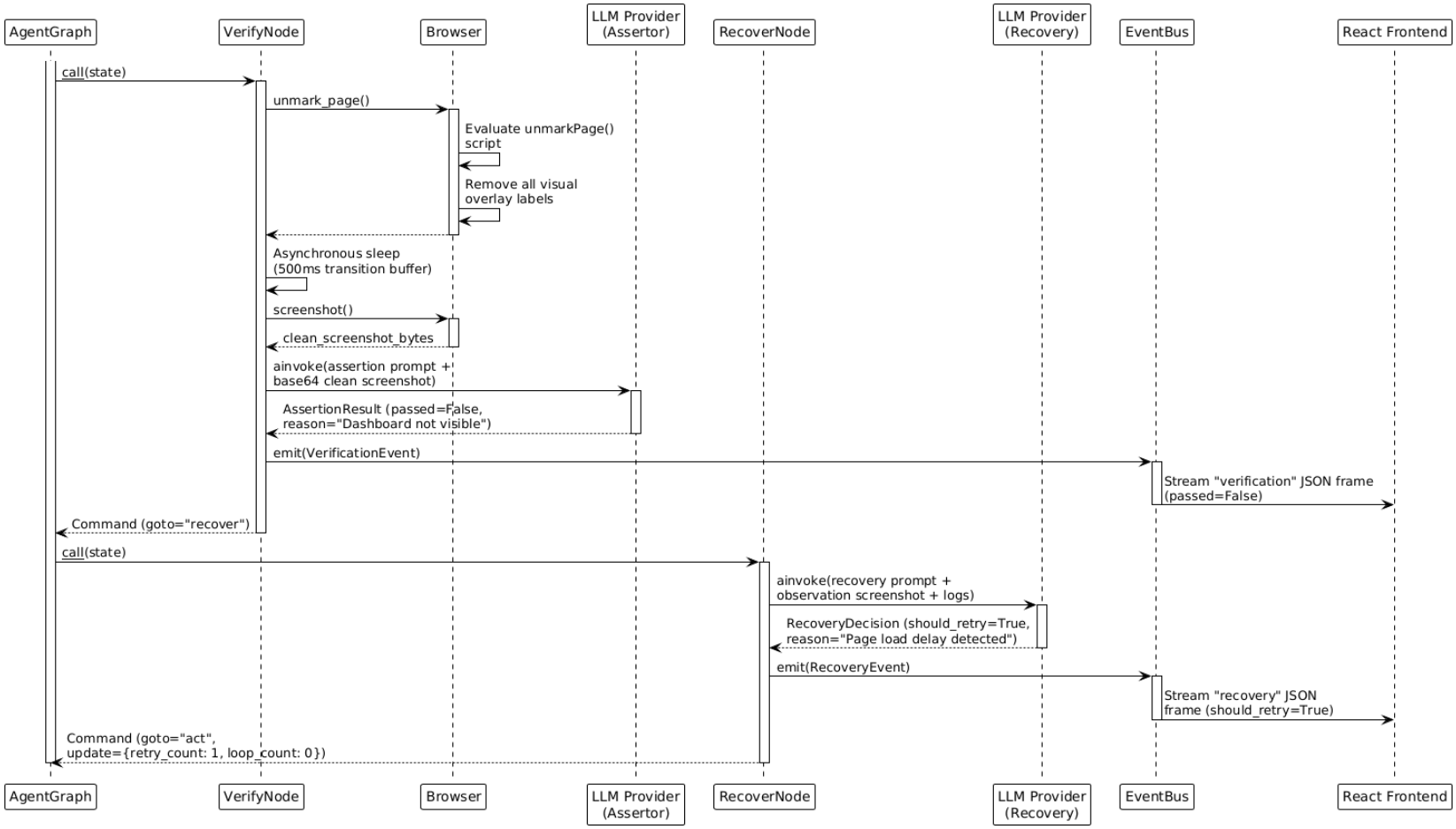


Figure 4.8: Step Verification, Failure Analysis, and Self-Healing Recovery Sequence Diagram

If the validation fails, the VerifyNode publishes a VerificationEvent to the EventBus, allowing the WebSocket handler to stream the failed outcome to the client, and returns a routing Command to transition execution to the RecoverNode. The RecoverNode evaluates whether the failure is recoverable or terminal. It constructs a recovery prompt containing the failed action, the expected assertion, the recent execution logs, and the previous observation screenshot, and invokes the recovery model.

The model returns a structured RecoveryDecision schema. If the recovery model determines that the failure was caused by a transient issue and the step's retry counter has not exceeded the limit, it decides to retry. The RecoverNode publishes a RecoveryEvent to the client, increments the retry_count, resets the active action loop_count to zero, and returns a routing Command pointing back to the ActNode to restart the step execution, which enables self-healing behavior. If the failure is terminal, it aborts the run and records the final failure verdict.

4.4 Database Entity-Relationship Diagram (ERD)

The relational database design of TestPilot is modeled using the Entity-Relationship Diagram (ERD) presented in Figure 4.9. The schema uses a fully normalized, local relational database structure to store project configurations, track execution progress, and persist structured telemetry data.

The database schema of TestPilot defines six distinct tables to represent system metadata and execution telemetry:

- projects: This table acts as the high-level logical workspace container. It contains a unique project identifier, the project name, an optional textual description, and a creation timestamp.
- test_cases: Stores natural language testing scripts. It contains a unique ID, parent project foreign key, optional external reference, test case name, binary execution type ('P' or 'N'), and target starting URL. Crucially, ordered steps are stored directly within the steps_json column as a serialized JSON array of objects, avoiding the overhead of a separate steps table.

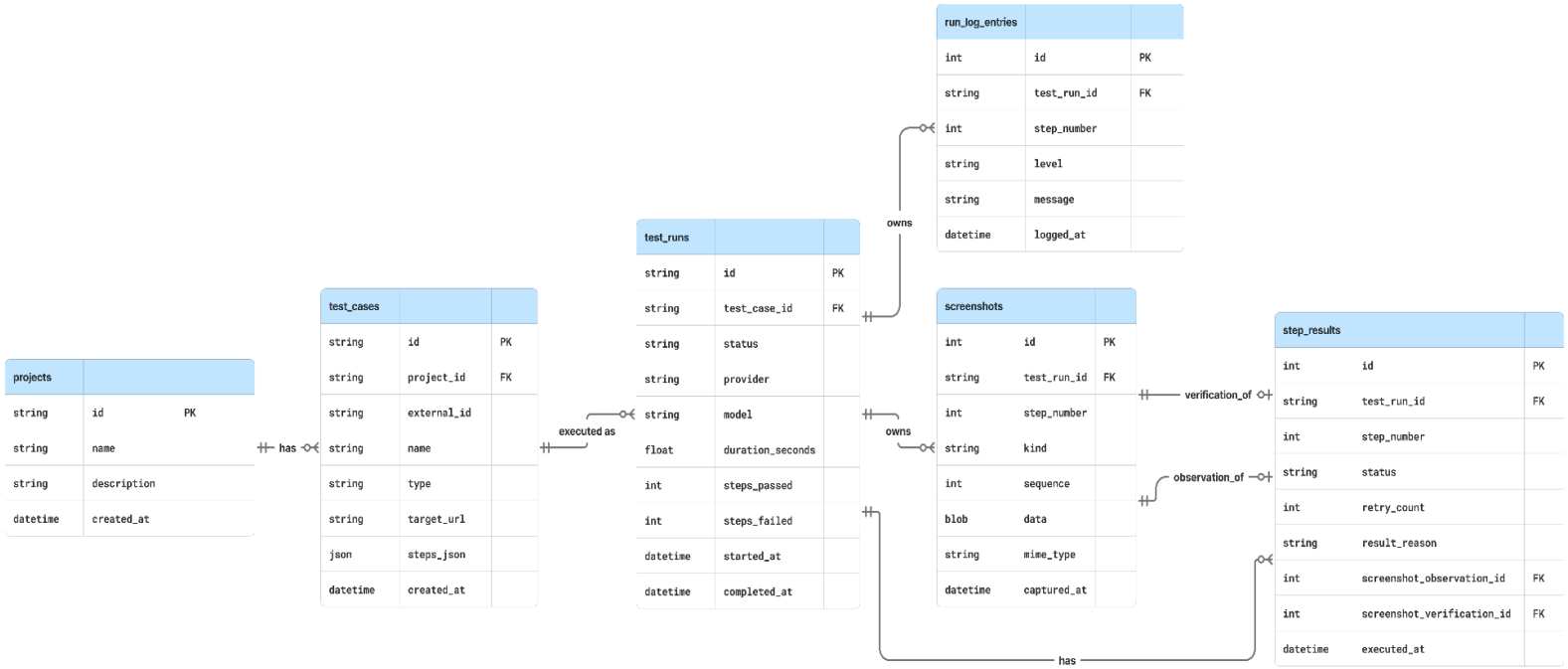


Figure 4.9: Database Entity-Relationship Diagram

- test_runs: This table records the execution metadata of each test run session. It tracks the run identifier, the parent test case foreign key, the execution status, the model provider, the language model name, the total duration in seconds, the count of passed steps, the count of failed steps, and the start and completion timestamps.
- screenshots: This table stores binary visual artifacts captured during execution. It holds a unique primary key, the parent test run foreign key, the active step number, the image kind (observation or verification), the sequence index, the MIME type, the creation timestamp, and the raw image data. The raw image data is stored as a binary large object (BLOB) within the database, which eliminates external filesystem dependencies.
- step_results: This table persists the outcomes of individual steps. It tracks the step result identifier, the parent test run foreign key, the step number, the verification status, the retry count, and a detailed result explanation string. Additionally, it stores two nullable foreign keys that link directly to the screenshots table to associate the step result with its specific observation and verification screenshots.
- run_log_entries: This table acts as the execution logging store. It contains a unique primary key, the parent test run foreign key, the step number, the logging severity level, the text message, and the logging timestamp.

The schema defines clear one-to-many (1:N) relationships across its tables, which are modeled using Crow's Foot notation in Figure 4.9. A single project has zero or many test cases, with the relationship enforced by the project_id foreign key in the test_cases table. Similarly, a single test case is executed as zero or many test runs, linked by the test_case_id foreign key in the test_runs table. Each test run session owns multiple step results, screenshots, and run log entries.

The relationships from the step_results table to the screenshots table are modeled as separate many-to-one (N:1) directed associations. A single step result record optionally references exactly one observation screenshot via the screenshot_observation_id foreign key and exactly one verification screenshot via the screenshot_verification_id foreign key.

Because SQLite does not enforce relational integrity by default, the database layer must actively enable foreign key constraints by executing the PRAGMA foreign_keys=ON command on every connection. This enforcement ensures that deleting a parent record (such as a project or a test case) triggers a cascading deletion down the dependency tree. Deleting a test run record will

automatically delete all its related screenshots, step results, and log entries, which prevents the accumulation of orphan records in the database.

4.5 User Interface Wireframes / Mockups

The user interface of TestPilot is implemented as a responsive, single-page web client built on React 18, Vite 8, and styled with Tailwind CSS 3. To maintain usability, the interface design uses a clean, grid-based administrative layout across three primary views.

This section presents mockups of the three main views, explaining their logical layout, component hierarchies, and interactive regions.

4.5.1 Projects Dashboard View

The Projects Dashboard serves as the entry point of the TestPilot application, presenting a structured overview of all testing projects. Figure 4.10 presents mockup of this dashboard interface.

The layout of the dashboard is divided into three distinct visual regions:

- The Navigation Bar: A persistent, top-aligned bar containing the TestPilot system title, a navigation link, and a prominent "+ New project" action button on the right. A status indicator on the left side of the title shows the connection state of the client-to-server WebSocket.
- The Dashboard Header: Displays a summary of the active workspaces, indicating the total number of projects registered in the database.
- The Project Grid: A responsive card-based grid that dynamically renders project statistics. Each card represents an independent Project entity. To support quick assessment, each card displays the project name, an optional text description, and a visual progress bar indicating the pass rate percentage. Below the progress bar, statistical counters display the total number of executed runs, passed runs, and failed runs. A project badge at the bottom of the

card indicates the number of test cases associated with the project, and a timestamp shows when the workspace was created. Each card also features a dropdown menu button on the top right for project deletion and configuration updates.

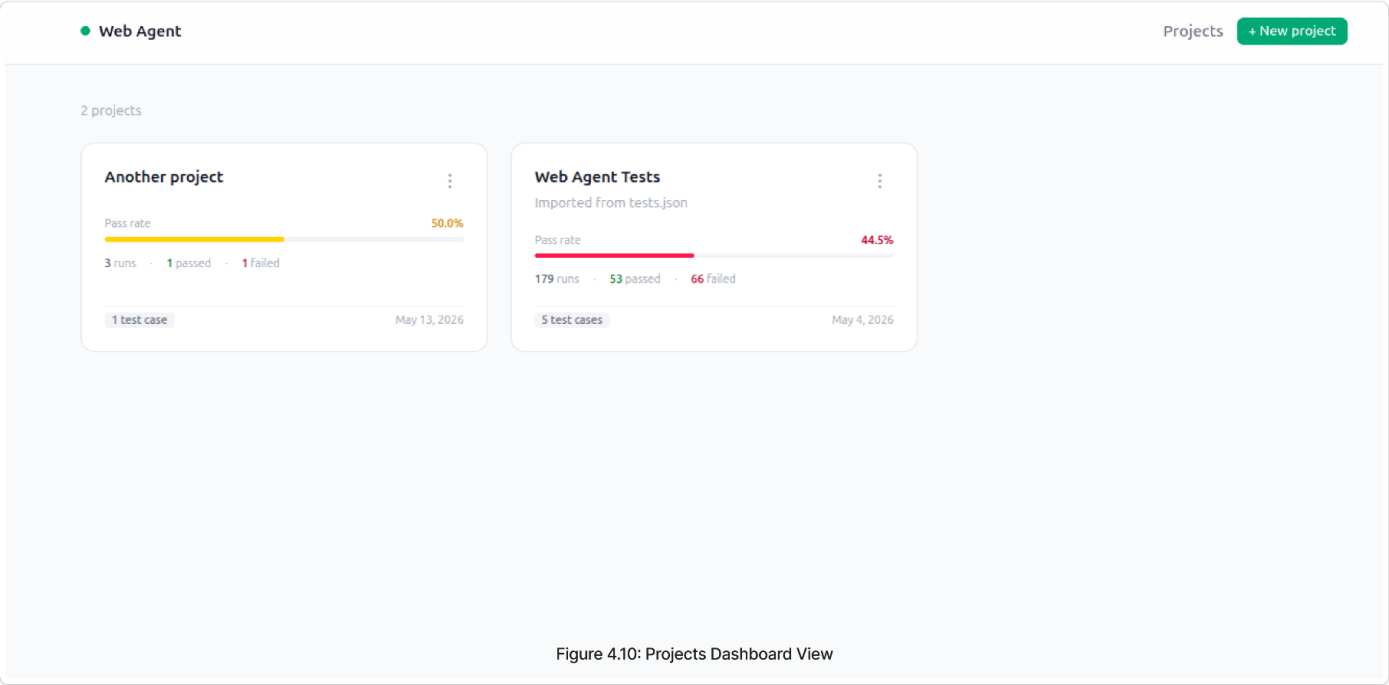


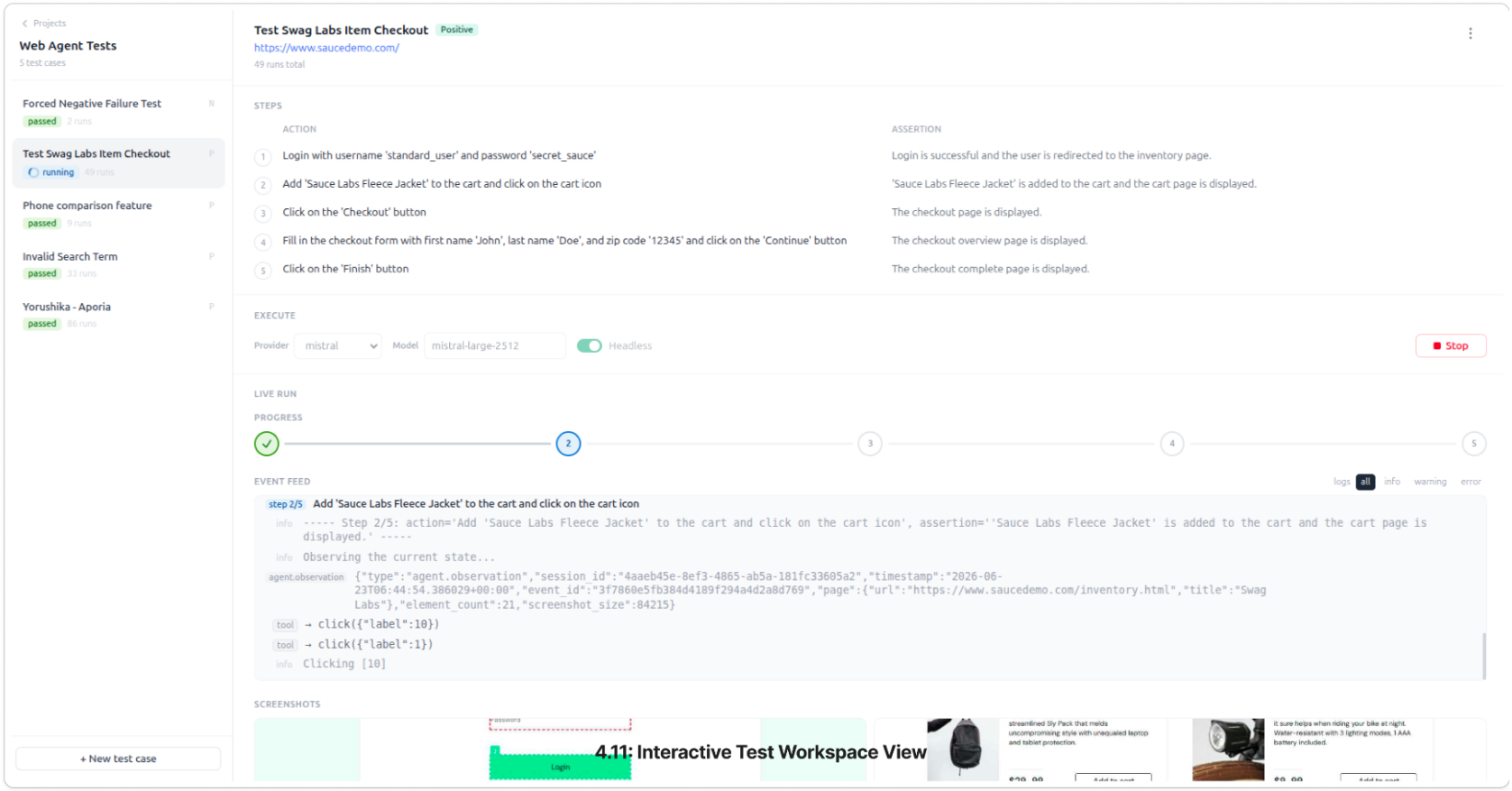
Figure 4.10: Projects Dashboard View

4.5.2 Interactive Test Workspace View

The Interactive Test Workspace is the primary control panel where users author, edit, and execute test cases. Figure 4.11 presents the high-fidelity mockup of this workspace.

The workspace is organized as a split-pane interface designed to support adjacent workflows:

- The Left Sidebar: Displays a directory of all test cases belonging to the active project, indicating the total test case count. Users can return to the main dashboard using a back navigation link. Below the header, a vertical list renders individual test cases, showing their name, last execution status (using a colored status badge), and total historical run count. A button is positioned at the bottom of the sidebar to allow users to add new test cases.



4.11: Interactive Test Workspace View

- The Right Main Panel (Execution Workspace): Displays the parameters of the currently active test case. The header displays the test case name, its target starting URL, and a badge indicating its test type (positive or negative). Below the header, a two-column steps table lists the sequential test steps, separating action instructions from expected assertions.
- The Execute Control Panel: Positioned below the steps table, this panel exposes execution parameters. It contains dropdown selection controls for the LLM provider and model, a toggle switch to enable or disable headless browser execution, and a control button (such as "Stop" or "Run") to manage active execution tasks.
- The Live Run Panel: When an execution run is active, this panel opens to show the live state. The "PROGRESS" section renders the execution progress as a horizontal node sequence. Completed steps display a checkmark, the active step is highlighted, and upcoming steps are greyed out. The "EVENT FEED" section streams real-time execution logs with filter tabs to isolate logging severities (all, info, warning, error). The feed displays the active step information, the underlying agent observation metadata containing the page URL and title, and the corresponding tool execution calls. The "SCREENSHOTS" section at the bottom displays the step-level screenshots captured during execution, rendering highlighted visual bounding boxes over interactive elements.

4.5.3 Historical Run Report View

The Historical Run Report View provides a read-only post-run analysis interface for completed test runs, which is used by developers and QA engineers to diagnose failures. Figure 4.12 presents the high-fidelity mockup of the report view.

The layout of the historical report is divided into three primary sections:

- The Navigation Header: Contains a back navigation button, the parent test case and run date identifier, an execution status badge on the right, and a deletion button.
- The Summary Metrics Cards: A horizontal row of four cards displaying the final execution parameters: the final execution status, the total execution duration in seconds, the provider used, and the percentage pass rate. Metadata below the cards shows the started and finished timestamps, the specific model used, and the final step completion ratio.

- The Step Results Table: Displays an ordered list of all executed steps. Each step card shows the step number, the action performed, the expected assertion, the step status, the retry count, and a detailed result explanation string returned by the assertor model. This explanation details the logical justification for the step's final status.
- The Screenshots Section: Displays side-by-side screenshots for each step. For each step, the left image shows the observation screenshot (with visual SoM label annotations overlaid on the interface, demonstrating how the model locates interactive elements), and the right image shows the verification screenshot (a clean page state with no labels, verifying the successful navigation and assertion outcomes).

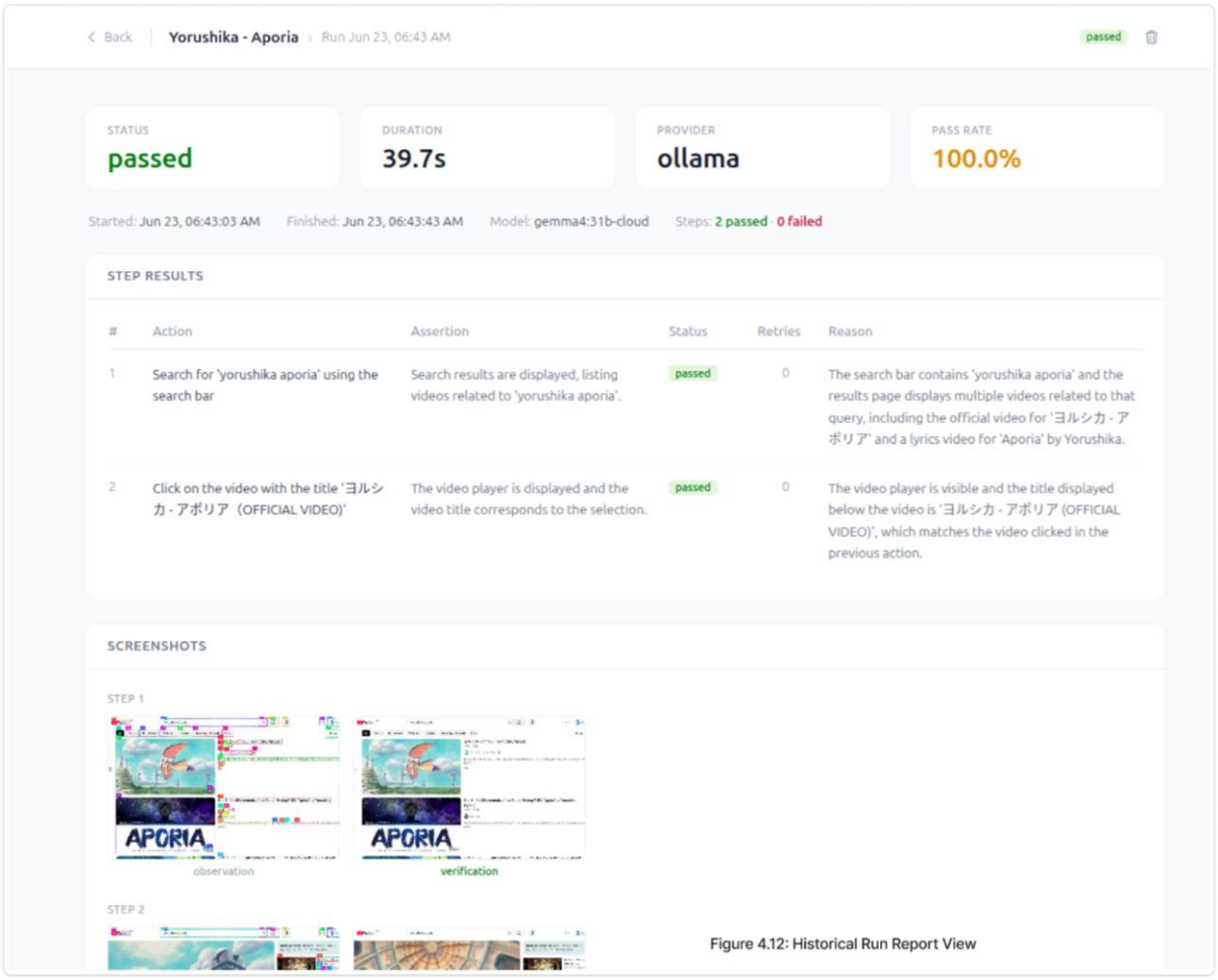


Figure 4.12: Historical Run Report View

4.6 Security Design Considerations

TestPilot requires security enforcement across data access, process execution, and network layers because it acts as an autonomous agent managing credentials, databases, and web browsers. This section details the security controls implemented to mitigate threats such as credential exposure, SQL injection, session cross-contamination, and system resource exhaustion.

4.6.1 API Key Lifecycle and In-Memory Containment

TestPilot interacts with external Large Language Model providers using API keys. To eliminate the risk of database leaks or disk-level compromise, the system uses a zero-persistence model. API keys are not stored in the SQLite database, written to configuration files, or persisted on disk.

During initialization, the Settings class reads API keys directly from the host environment variables. These keys are stored inside volatile system memory within a private, in-memory dictionary (dict[str, str]). When instantiating models, the service retrieves keys from this dictionary and passes them directly to the model constructor. Consequently, credentials exist only within the isolated memory space of the active process and are purged when the application terminates. This design ensures that a compromised database file does not leak API keys.

4.6.2 Injection Prevention and Server-Side Validation

To prevent SQL injection, TestPilot prohibits raw SQL queries. All database transactions run through the SQLAlchemy ORM. The ORM compiles Python data operations into parameterized SQL queries, which decouples the query structure from the data inputs. Parameterization ensures that the database engine treats all incoming inputs strictly as literal values instead of executable code.

Additionally, the FastAPI server enforces input validation at the API boundary using Pydantic schemas. Every incoming request payload, such as project configuration parameters or test steps,

is parsed and validated against type definitions and string constraints before it reaches the persistence layer.

4.6.3 Execution Context and Session Isolation

During test execution against dynamic web applications, the browser is exposed to untrusted website content such as scripts, cookies, and local storage data. To prevent cross-test contamination and data leaks, the Playwright browser controller does not reuse browser contexts.

For every execution run, the controller instantiates an isolated, ephemeral browser context. This context operates in a clean state, which prevents the sharing of cookies, cached scripts, session tokens, and local storage properties between runs. When a test run completes, aborts, or fails, the system closes the browser context and purges its memory state. This teardown prevents target application data from persisting on the host machine.

4.6.4 Asynchronous Task and Process Containment

Autonomous web agents can cause infinite loops, process accumulation, and resource exhaustion, which degrades host machine performance. TestPilot implements process containment controls to manage system resources.

To prevent duplicate runs for the same test case, the ExecutionManager enforces concurrency limits using an asynchronous lock. Furthermore, the LangGraph state machine enforces an execution loop limit and a retry limit. These boundaries prevent the agent from entering infinite planning loops. When a run is canceled, the ExecutionManager triggers a cancellation handler that terminates the corresponding task, closes the Playwright Chromium process, and releases the execution lock. This cleanup prevents headless browser processes from remaining as zombie processes on the server.

Chapter 5: Implementation

This chapter documents the implementation phase of TestPilot, describing the environment configurations, code structures, and implementation details of the active system components. It details the runtime dependencies across the backend and frontend frameworks, deconstructs the state transitions of the LangGraph agent, and provides concrete visual samples of the running application.

5.1 Implementation Environment and Setup

The software development, local deployment, and verification of TestPilot require a structured runtime environment across both client and server layers. The system is designed to run locally or inside containerized virtual spaces, which prevents operating system dependencies from polluting the host configuration.

5.1.1 Operating System and Environment Sandbox

TestPilot supports multi-platform execution, running natively on Windows 10+ and Ubuntu operating systems. To isolate Python packages and Node.js dependencies, the development environment is managed using sandboxed package environments.

On the backend, Python virtual environments (venv) are used to isolate libraries and prevent dependency version conflicts. On the frontend, Node Version Manager (nvm) is used to enforce a standard Node.js runtime environment across all development workstations.

5.1.2 Backend Runtime and Dependency Configurations

The backend server is powered by a Python 3.13 runtime. This runtime was selected for its performance optimizations and stable support for asynchronous execution contexts (asyncio). The ASGI server wrapper is Uvicorn (0.40.0), which runs the asynchronous FastAPI application.

The backend architecture depends on a specific set of libraries to coordinate planning and data storage. These libraries are managed through a standard requirements.txt file and are detailed below:

- FastAPI (0.128.0): Provides the asynchronous web routing framework used to manage REST endpoints and coordinate bidirectional WebSocket channels.
- LangGraph (1.1.0): Serves as the state orchestration library used to compile the agent loop as a stateful graph.
- LangChain Core (1.4.0): Provides the foundational message classes, prompt templates, and chat model interfaces used to integrate with language model APIs.
- Playwright (1.50.0): Drives headless Chromium browser contexts, captures page screenshots, and executes custom visual element-labeling scripts.
- SQLAlchemy (2.0.40): An ORM used to define schema structures and execute parameterized relational database queries.
- SQLite (3.45.0): A local, serverless relational database engine used to store project configurations, test run histories, and binary screenshots.
- Pydantic (2.12.0): Enforces data parsing and strict type-validation constraints at the REST API boundary.
- Pillow (12.0.0): A Python imaging library used to perform LANCZOS visual resizing and apply JPEG compression on captured page screenshots.

During the initial installation phase, the browser binaries must be compiled and registered locally by running the Playwright driver installation command (playwright install chromium).

5.1.3 Frontend Runtime and Dependency Configurations

The frontend web client is executed within a Node.js v20.11.0 LTS environment using the npm v10.2.4 package manager. The client-side application is bundled and served using Vite 8 (^8.0.10), which provides an asynchronous build pipeline and proxy configurations to route frontend requests to the local FastAPI port.

The frontend uses the following programmatic packages to construct the user interface:

- React (18.3.1) and React DOM (18.3.1): Provide the component-based UI framework and concurrent rendering engine.
- React Router DOM (6.26.2): Manages client-side route registration, view transitions, and search parameter parsing.
- TanStack React Query (5.56.2): Provides server-state caching, queries orchestration, mutations handling, and cache invalidation workflows.
- Tailwind CSS (3.4.10): A utility-first styling framework used to design the grid layouts and color-coded status badges of the client.

5.2 Key Implementation Details

This section deconstructs the programmatic algorithms, state management modules, and major engineering design decisions that govern the runtime behavior of TestPilot.

5.2.1 Element Discovery and Set-of-Marks Overlay Algorithm

The visual grounding system is implemented in mark_page.js to identify visible, interactive elements and overlay high-contrast labels without altering the page layout. First, the script queries the DOM using a specific CSS selector, INTERACTIVE_SELECTOR, which targets native interactive HTML tags and ARIA roles. It then uses document.createTreeWalker() with a custom filter to locate non-standard interactive elements, such as those containing inline onclick handlers, CSS cursor: pointer styles, or interactive ARIA roles.

For each candidate, the script verifies visibility by checking that its CSS properties do not equal display: none or visibility: hidden. The bounding box is validated by getVisibleRect(), which loops over element.getClientRects() and verifies that document.elementFromPoint() at the element's center coordinates returns the element itself or its descendants, which ensures that the element is not obscured by overlapping overlays. The script discards elements smaller than 20 square pixels or those positioned more than 500 pixels off-screen.

It draws a dashed label outline on the viewport using high z-index values (2147483647) and applies pointer-events: none to prevent overlays from intercepting clicks. Finally, the script assigns a custom attribute, data-mark, to each identified DOM element, returning the serialized element metadata (role, tag name, state, and description) to the ActNode, which allows Playwright to locate elements using the attribute selector page.locator("[data-mark='label']").

5.2.2 DOM Stability Detection Algorithm

To prevent the agent from planning actions or evaluating assertions on empty or partially rendered pages, the browser controller implements a deterministic polling algorithm named _wait_stable(). The stability detection loop is executed automatically after every browser navigation, page reload, or tool action.

The algorithm runs inside an asynchronous polling loop, measuring the total count of all DOM nodes (via document.getElementsByTagName('*').length) and the text length of the document body (via document.body.innerText.length) at an interval of 300 milliseconds. If both values remain identical across consecutive interval measurements, the page is classified as stable, and the loop terminates. If either value changes, indicating that the page is still loading resources, the stability timer resets, repeating this verification process up to a maximum safety timeout threshold of 3000 milliseconds.

5.2.3 Major Engineering Decisions

During the development of TestPilot, three major architectural decisions were made to improve database performance, data portability, and execution stability:

- SQLite BLOB Storage over Filesystem Storage: The original design proposed storing captured page screenshots as individual files on the local filesystem. This was changed to a consolidated database model where screenshots are stored directly as binary large objects (BLOB data) in the screenshots table, and execution logs are stored as individual database records in the run_log_entries table. This monolithic database design guarantees

transactional atomicity (as screenshot writes are committed alongside step outcomes), eliminates filesystem synchronization errors, and simplifies application containerization since the entire state is represented within a single testpilot.db file.

- JSON Steps Column over Independent Steps Table: To represent the sequential steps of a test case, the database uses a single JSON-serialized column named steps_json inside the test_cases table, rather than a separate child table. This JSON serialization avoids the database overhead of managing foreign key cascades, sequence index updates, and multiple transactional round-trips when steps are reordered, inserted, or duplicated.
- Asynchronous Event Decoupling: Because storing large binary screenshots can introduce significant database write latency, TestPilot decouples the active test execution from database transactions. The ExecutionService executes the LangGraph agent and publishes telemetry events to an in-memory, asynchronous EventBus without waiting for database confirmation. The DatabaseListener subscribes to this bus, captures the events, and writes the records, screenshots, and logs to SQLite in the background using isolated database sessions, which ensures that database transaction latency never blocks the browser automation loop.

5.3 Output Samples and Data Payloads

While Section 4.5 presented the visual layouts of the React interface, this section documents the underlying data outputs, serialization formats, and asynchronous communication payloads generated by TestPilot during runtime execution. This complete documentation covers the JSON serialization format used for data portability, the asynchronous WebSocket event frames used for real-time progress streaming, and the database-level logging records.

5.3.1 JSON Test Case Portability Payload

To support test case persistence, import, and export capabilities, TestPilot represents test cases as structured JSON objects. This design allows users to share test cases across different project

workspaces or export them as backup files. Listing 5.1 presents an exported JSON payload of a five-step test case designed for an e-commerce application.

```
{
  "project_id": "8f39b1a2-7c4d-4e9e-bc81-2a101f33605a",
  "name": "Test Swag Labs Item Checkout",
  "type": "P",
  "target_url": "https://www.saucedemo.com/",
  "external_id": "TC-SWAG-001",
  "steps": [
    {
      "action": "Login with username 'standard_user' and password 'secret_sauce'",
      "assertion": "Login is successful and the user is redirected to the inventory
page."
    },
    {
      "action": "Add 'Sauce Labs Fleece Jacket' to the cart and click on the cart
icon",
      "assertion": "'Sauce Labs Fleece Jacket' is added to the cart and the cart
page is displayed."
    },
    {
      "action": "Click on the 'Checkout' button",
      "assertion": "The checkout page is displayed."
    },
    {
      "action": "Fill in the checkout form with first name 'John', last name 'Doe',
and zip code '12345' and click on the 'Continue' button",
      "assertion": "The checkout overview page is displayed."
    },
    {
      "action": "Click on the 'Finish' button",
      "assertion": "The checkout complete page is displayed."
    }
  ]
}
```

This JSON payload is parsed by the FastAPI backend, validated against the Pydantic TestCaseCreate schema, and converted into a relational database record. The steps array is serialized directly into the database test_cases table under the steps_json column. This database design optimization eliminates the overhead of managing a separate database table for individual steps while preserving sequential integrity.

5.3.2 WebSocket Event Frame Payload

During test execution, the backend serializes the active run state and streams it to the web client using asynchronous WebSocket event frames as specified in FR-8. Listing 5.2 presents a raw, serialized agent.observation JSON frame emitted by the EventBus and transmitted over the active WebSocket channel during Step 2 of the execution.

```
{
  "type": "agent.observation",
  "session_id": "4aaeb45e-8ef3-4865-ab5a-181fc33605a2",
  "timestamp": "2026-06-23T13:29:44.543Z",
  "event_id": "3f7860e5fb384d4189f294a2da28d769",
  "payload": {
    "step_index": 2,
    "page": {
      "url": "https://www.saucedemo.com/inventory.html",
      "title": "Swag Labs"
    },
    "element_count": 21
  }
}
```

Upon receiving this frame, the useRunWebSocket hook in the React frontend parses the payload. The client-side state manager uses the payload.step_index to update the progressive step tracker, and the payload.page object to update the active URL and title displays, which ensures that the user interface matches the backend execution state within 100 milliseconds of being received via WebSockets.

5.3.3 Raw Execution Log Output

The system records asynchronous runtime logging messages to support debugging and trace verification. These log entries are captured during execution and persisted as discrete records in the run_log_entries table. Listing 5.3 presents a sample block of raw, sequential log outputs retrieved from the database for a completed test run.

```
2026-05-19T06:58:04.441449 - INFO - Starting browser...
2026-05-19T06:58:05.175862 - INFO - Browser ready
2026-05-19T06:58:05.186641 - INFO - Navigating to https://www.saucedemo.com/
2026-05-19T06:58:06.708534 - INFO - ----- Step 1/5: action='Login with username
'standard_user' and password 'secret_sauce'', assertion='Login is successful and
the user is redirected to the inventory page.' -----
2026-05-19T06:58:06.735739 - INFO - Observing the current state...
2026-05-19T06:58:14.395089 - INFO - Filling [0] with 'standard_user...'
2026-05-19T06:58:14.697168 - INFO - Filling [1] with 'secret_sauce...'
2026-05-19T06:58:14.951752 - INFO - Clicking [2]
2026-05-19T06:58:15.378366 - INFO - Verifying: Login is successful and the user is
redirected to the inventory page.
2026-05-19T06:58:18.615014 - INFO - Step 1 passed: The screenshot shows the
'Products' page of Swag Labs, which is the inventory page. This confirms that the
login action was successful and the user was redirected as expected.
2026-05-19T06:58:18.624786 - INFO - ----- Step 2/5: action='Add 'Sauce Labs Fleece
Jacket' to the cart and click on the cart icon', assertion=''Sauce Labs Fleece
Jacket' is added to the cart and the cart page is displayed.' -----
```

These log rows are retrieved through the GET /api/test-runs/{id}/logs endpoint, which allows the React frontend to display a complete terminal feed of the test's asynchronous execution operations directly in the user interface.

Chapter 6: Testing and Evaluation

This chapter documents the testing, evaluation, and benchmarking phase of TestPilot. To verify the functional accuracy, performance parameters, and operational stability of the proposed agent, we executed an empirical evaluation using the standardized, public SauceLabs Swag Labs sandbox. This empirical test is followed by an analytical comparison that maps TestPilot's structural design against the published academic baselines of SeeAct-ATA and PinATA [1].

6.1 Evaluation Setup and Test Suite

The large-scale evaluation of autonomous test agents relies on the public WebArena benchmark, which requires over 120 gigabytes of local storage to mirror complete web applications. Because local hardware and storage constraints prevented the execution of these massive server images, we established a reproducible empirical testing suite using the public, stable SauceLabs Swag Labs (https://www.saucedemo.com/) sandbox. This sandbox represents a standard e-commerce platform. It contains multi-step transaction flows, stateful shopping carts, form validation inputs, and authentication boundaries.

We designed a suite of natural language test cases to evaluate the execution accuracy of TestPilot under positive and negative boundaries. Table 6.1 presents the configurations and validation objectives of this test suite.

Test ID	Test Case Name	Type	Validation Objective
TC-1	Valid User Login	Positive	Verify successful authentication and redirection to the product inventory page.
TC-2	Invalid Password Login	Negative	Verify that the system handles authentication failure and displays the correct error message.
TC-3	Locked Out User Login	Negative	Verify that the system identifies the locked-out user state and halts execution.
TC-4	Add Single Item to Cart	Positive	Verify that clicking the add-to-cart button updates the cart badge count.

TC-5	Add Multiple Items to Cart	Positive	Verify that adding several products updates the shopping cart badge with the correct cumulative count.
TC-6	Remove Item from Cart	Positive	Verify that removing an item from the cart page updates the cart state and decreases the badge count.
TC-7	Complete Standard Checkout	Positive	Verify the complete purchase workflow, including cart access, checkout form submission, and checkout completion.
TC-8	Broken Visuals Complete Standard Checkout	Positive	Verify the complete purchase workflow with broken visuals, including cart access, checkout form submission, and checkout completion.
TC-9	Checkout Missing Information	Negative	Verify that the system detects form validation errors when submitting an incomplete checkout form.
TC-10	Cart State Persistence	Positive	Verify that the added items persist in the shopping cart when navigating back to the inventory page.
TC-11	Reset Application State	Positive	Verify that clicking the reset sidebar option clears all active cart selections and resets the interface.

Table 6.1: SauceLabs Test Cases Suite

6.3 Design-Level Baseline Comparison

The performance of TestPilot can be evaluated analytically by comparing its architectural features with the published baseline results of PinATA and SeeAct-ATA from the reference literature [1]. Table 6.2 outlines the structural design comparison between these systems.

Architectural Feature	SeeAct-ATA	PinATA	TestPilot
Orchestration Model	Single Prompt	Multi-Agent Orchestrator	LangGraph State Machine
Intermediate Assertions	Inferred by LLM	Dedicated Assertor	Dedicated Assertor Node
Structured Output Schema	Text parsing	JSON parser	Pydantic Schema Enforcement
Error Recovery Mechanism	None	Orchestrator Retry	Orchestrator Retry
Step-Level Tracing	No	Yes	Yes

Table 6.2: Design Comparison Between TestPilot and Baselines

TestPilot's specific design choices are structurally engineered to resolve the primary failure modes documented in existing research. In the baseline evaluations of Chevrot et al., agents frequently encountered "Scenario Conformity" errors because a single-prompt or loosely coordinated multi-agent model would drift from the sequential steps of the test case [1]. TestPilot resolves this by compiling the workflow as a stateful LangGraph state machine, which guarantees strict, sequential node transitions.

Furthermore, baseline agents exhibited high assertion verifiability error rates because the visual markings used for action selection remained on the page during assertion checking. TestPilot's VerifyNode eliminates this issue by executing an explicit unmarkPage() script to capture a clean, unannotated screenshot before verification. This process ensures that visual validation remains objective and accurate.

6.4 Threats to Validity

The primary threat to the external validity of this evaluation is the deployment restriction of our testing environment. The physical deployment of the full, large-scale WebArena offline benchmark applications requires over 120 gigabytes of disk storage and significant local hosting

resources. These requirements are typical of enterprise servers but remain unavailable on standard development workstations.

This operational limit was successfully addressed by executing our empirical validation on the public, stable SauceDemo e-commerce sandbox. This allowed us to verify the complete functionality of the browser controller, the event streaming, the state machine transitions, and the database persistence layers.

Another potential threat is the network latency and rate-limiting behavior of external Large Language Model APIs. This threat is addressed by configuring local retry thresholds and implementing local exception-catching blocks in our recovery state nodes.

Chapter 7: Conclusion and Future Work

This chapter concludes the final report for TestPilot. It provides a summary of the achievements, documents the core software engineering lessons learned during the development lifecycle, and outlines proposed future enhancements to address the identified limitations of the prototype.

7.1 Summary of Achievements

The major achievements completed during the development of TestPilot include:

- Design of a Modular Stateful Architecture: We successfully engineered the core agent orchestration layer using a stateful graph on LangGraph. This architecture partitions test step execution into five independent, specialized nodes: InitStepNode, ActNode, VerifyNode, RecoverNode, and SequentialToolExecutor. This separation of concerns prevents the planning, execution, and verification phases from polluting one another, which stabilizes the test execution flow.
- Implementation of Selector-Free Grounding: We developed a visual element-discovery and DOM serialization system. By injecting custom JavaScript modules into Chromium via Playwright, the system calculates element bounding boxes, generates a high-contrast SoM visual overlay on screenshots, and assigns unique, temporary element labels. This allows the agent to interact with target pages without relying on hardcoded CSS selectors or XPath locators.
- Development of a Responsive Web Interface: We implemented a responsive single-page web client using React 18, Tailwind CSS 3, and Vite 8. The client communicates with the FastAPI backend using stateless REST APIs for configuration management and persistent WebSockets for live execution streaming. The frontend allows users to manage projects, edit natural language test cases, and monitor execution progress, logs, and screenshots in real time.

7.2 Lessons Learned

The systematic design, implementation, and testing of TestPilot yielded several critical engineering insights regarding autonomous web agents and software testing:

- Decoupling Execution from Verification: In early prototyping, the system combined action execution and assertion verification within a single node. This design led to high validation error rates because the language model would frequently verify its own actions incorrectly. Decoupling the validator into a dedicated VerifyNode (the assertor) that evaluates page states on a clean, unmarked screenshot was essential to ensure objective verification results.
- The Crucial Role of Structured Schemas: Attempting to parse raw, free-form text outputs from language models consistently introduced formatting exceptions, which caused state graph crashes. Enforcing structured output schemas using Pydantic models for the assertor (AssertionResult) and the recovery node (RecoveryDecision) completely resolved these parsing errors and stabilized the state transitions.
- Limitations of Purely Visual Grounding: While the SoM visual overlay was highly effective for element localization, purely visual evaluations are vulnerable to color contrast conflicts and hidden elements. Combining visual screenshots with filtered structural accessibility tree metadata is necessary to ensure reliable element localization on modern, complex single-page applications.

7.3 Future Enhancements

To build upon the current prototype of TestPilot and resolve its identified operational limitations, the following future research and development directions are proposed:

- Local Vision-Language Model Fine-Tuning: The current implementation of TestPilot depends entirely on external commercial API providers, which introduces latency and token consumption costs. Future work will explore the integration of smaller, open-source Vision-Language Models (such as CogAgent or UI-TARS) hosted locally on private server infrastructure. Fine-tuning these models directly on historical web testing execution traces

would reduce network latency, eliminate API subscription costs, and ensure absolute data privacy.

- Ingestion of Network and Console Telemetry: The active verification node currently evaluates page states purely through visual inspections of screenshots. Future iterations of TestPilot will expand the observation data to ingest active HTTP network traffic logs and browser console errors captured via Playwright. This will enable the assertor to detect silent backend exceptions, failed API requests, and JavaScript execution errors that do not manifest visually on the user interface.
- Data-Driven Parameterization: To support standard industry testing practices, the testcase engine should be extended to support data-driven testing. This enhancement will enable users to upload CSV or JSON data files to the React frontend, allowing TestPilot to execute a single natural language test case multiple times using different variable inputs (such as varied username credentials or checkout values) mapped dynamically from the data file.
- Parallel Execution Scaling: The current ExecutionManager uses a lock to prevent concurrent executions of the same test case, and executes runs sequentially. Future iterations will focus on refactoring the execution layer to support parallel browser workers. By spawning independent, sandboxed container instances, TestPilot will be able to execute multiple different test runs in parallel, which will reduce the total time required to validate large, detailed regression suites.

References

[1] A. Chevrot, A. Vernotte, J.-R. Falleri, X. Blanc, and B. Legeard, "Are autonomous web agents good testers?," arXiv preprint arXiv:2504.01495, Apr. 2025.

[2] S. Zhou et al., "WebArena: A realistic web environment for building autonomous agents," arXiv preprint arXiv:2307.13854, Jul. 2023.

[3] Applitools, "Applitools | AI-automated compliance testing." [Online]. Available: https://applitools.com/ (accessed Feb. 7, 2026).

[4] Functionize, "Enterprise AI test automation platform with QA agents." [Online]. Available: https://www.functionize.com/ (accessed Feb. 7, 2026).

[5] Testim, "AI-powered automated UI and functional testing." [Online]. Available: https://www.testim.io/ (accessed Feb. 7, 2026).

[6] W. Chen, H. Cao, and X. Blanc, "An improving approach for DOM-based web test suite repair," in *Web Engineering*, M. Brambilla, R. Chbeir, F. Frasincar, and I. Manolescu, Eds. Biarritz, France: Springer International Publishing, 2021, pp. 372–387, doi: 10.1007/978-3-030-74296-6_29.

[7] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2Web: Towards a generalist agent for the web," arXiv preprint arXiv:2306.06070, 2023, doi: 10.48550/arXiv.2306.06070.

[8] S. Di Meglio and L. L. L. Starace, "Towards predicting fragility in end-to-end web tests," in *Proc. 28th Int. Conf. Eval. Assess. Softw. Eng. (EASE '24)*, Salerno, Italy, 2024, pp. 387–392, doi: 10.1145/3661167.3661179.

[9] R. Feldt, S. Kang, J. Yoon, and S. Yoo, "Towards autonomous testing agents via conversational large language models," arXiv preprint arXiv:2306.05152, 2023, doi: 10.48550/arXiv.2306.05152.

[10] B. García, J. M. del Alamo, M. Leotta, and F. Ricca, "Exploring browser automation: A comparative study of Selenium, Cypress, Puppeteer, and Playwright," in *Quality of Information and Communications Technology*, A. Bertolino, J. P. Faria, P. Lago, and L. Semini, Eds. Pisa, Italy: Springer Nature Switzerland, 2024, pp. 142–149, doi: 10.1007/978-3-031-70245-7_10.

[11] I. Gur, H. Furuta, A. Huang, M. Safdari, Y. Matsuo, D. Eck, and A. Faust, "A real-world WebAgent with planning, long context understanding, and program synthesis," arXiv preprint arXiv:2307.12856, 2023.

[12] M. Hammoudi, G. Rothermel, and A. Stocco, "WATERFALL: An incremental approach for repairing record-replay tests of web applications," in *Proc. 24th ACM SIGSOFT Int. Symp. Found. Softw. Eng. (FSE 2016)*, Seattle, WA, USA, 2016, pp. 751–762, doi: 10.1145/2950290.2950294.

[13] M. Hammoudi, G. Rothermel, and P. Tonella, "Why do record/replay tests of web applications break?," in *2016 IEEE Int. Conf. Softw. Test., Verif. Valid. (ICST)*, Chicago, IL, USA, 2016, pp. 180–190, doi: 10.1109/ICST.2016.16.

[14] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "WebVoyager: Building an end-to-end web agent with large multimodal models," arXiv preprint arXiv:2401.13919, 2024, doi: 10.48550/arXiv.2401.13919.

[15] H. Lai, X. Liu, I. L. Iong, S. Yao, Y. Chen, P. Shen, H. Yu, H. Zhang, X. Zhang, Y. Dong, and J. Tang, "AutoWebGLM: A large language model-based web navigating agent," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining (KDD '24)*, Barcelona, Spain, 2024, pp. 5295–5306, doi: 10.1145/3637528.3671620.

[16] M. Leotta, B. García, F. Ricca, and J. Whitehead, "Challenges of end-to-end testing with Selenium WebDriver and how to face them: A survey," in *2023 IEEE Conf. Softw. Test., Verif. Valid. (ICST)*, Dublin, Ireland, 2023, pp. 339–350, doi: 10.1109/ICST57152.2023.00039.

[17] M. Leotta, A. Stocco, F. Ricca, and P. Tonella, "Reducing web test cases aging by means of robust XPath locators," in *2014 IEEE Int. Symp. Softw. Reliab. Eng. Workshops*, Naples, Italy, 2014, pp. 449–454, doi: 10.1109/ISSREW.2014.17.

[18] M. Leotta, A. Stocco, F. Ricca, and P. Tonella, "Using multi-locators to increase the robustness of web test cases," in *2015 IEEE 8th Int. Conf. Softw. Test., Verif. Valid. (ICST)*, Graz, Austria, 2015, pp. 1–10, doi: 10.1109/ICST.2015.7102611.

[19] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE '24)*, Lisbon, Portugal, 2024, doi: 10.1145/3597503.3639180.

[20] Z. Long, G. Wu, X. Chen, W. Chen, and J. Wei, "WebRR: Self-replay enhanced robust record/replay for web application testing," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE 2020)*, Virtual Event, USA, 2020, pp. 1498–1508, doi: 10.1145/3368089.3417069.

[21] V. Nguyen, T. To, and G.-H. Diep, "Generating and selecting resilient and maintainable locators for Web automated testing," *Softw. Test., Verif. Reliab.*, vol. 31, no. 3, p. e1760, 2021, doi: 10.1002/stvr.1760.

[22] A. Stocco, R. Yandrapally, and A. Mesbah, "Visual web test repair," in *Proc. 2018 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE 2018)*, Lake Buena Vista, FL, USA, 2018, pp. 503–514, doi: 10.1145/3236024.3236063.

[23] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, p. 186345, Dec. 2024, doi: 10.1007/s11704-024-40231-1.

[24] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, "Set-of-Mark prompting unleashes extraordinary visual grounding in GPT-4V," arXiv preprint arXiv:2310.11441, 2023, doi: 10.48550/arXiv.2310.11441.

[25] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "GPT-4V(ision) is a generalist web agent, if grounded," in *Proc. 41st Int. Conf. Mach. Learn., Proc. Mach. Learn. Res.*, vol. 235, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds. Vienna, Austria: PMLR, 2024, pp. 61349–61385.